Received: 25 December 2020 | Accepted: 27 December 2020

DOI: 10.1002/net.22022

SPECIAL ISSUE ARTICLE

WILEY

Modeling the flying sidekick traveling salesman problem with multiple drones

Mauro Dell'Amico | **Roberto Montemanni** | **Stefano Novellani**

DISMI, University of Modena and Reggio Emilia, Reggio Emilia, Italy

Correspondence
Stefano Novellani, DISMI, University of Modena and Reggio Emilia, 42122 Reggio Emilia, Italy.
Email: stefano.novellani@unimore.it

Abstract

This article considers a version of the flying sidekick traveling salesman problem in which parcels are delivered to customers by either a truck or a set of identical flying drones. The flights of the drones are limited by the battery endurance and each flight is made of a launch, a service to a customer, and a return: launch and return must happen when the truck is stationary. These operations require time and when multiple drones are launched and/or collected at the same node, their order becomes relevant. The proposed model takes into account the order of these operations as a scheduling problem, because ignoring it could lead to infeasible solutions in the reality due to the possible exceeding of the drones endurance. We propose a set of novel formulations for the problem that can improve the size of the largest instances solved in the literature. We provide a comparison among the formulations, between the multiple drones solutions and the single drone ones, and among different variants of the model.

KEYWORDS

aerial drones, branch-and-cut, drone scheduling, formulations, MILP, parcel delivery, UAV

1 | INTRODUCTION

Retail e-commerce sales worldwide doubled from 1336 to 3535 billion US dollars from 2014 to 2019 and are forecasted to grow to 6542 billions in 2023 (see [39]). The e-commerce boom requires new investments and ideas in parcel delivery; indeed, a recent Boston Consulting Group study revealed that parcel and express delivery startups founding has increased by 20 times between 2014 and 2016 (see [43]). Another recent study by McKinsey (see [19]) shows that the request for fast parcel delivery is relevant: 20% to 25% of customers are willing to pay more for same-day delivery and 2% for instant delivery. The same study confirms that this request could be fulfilled by autonomous vehicles and forecasts that autonomous vehicles will deliver 80% of all parcels in the next ten years. Among autonomous vehicles there are aerial drones, also called unmanned aerial vehicles (UAV) or, in the following, just drones. The most important e-commerce and parcel logistics companies have considered, implemented, or are willing to implement parcel delivery by drones. We can mention Alibaba, Alphabet, Amazon, DHL, and JD.com, among the others. Moreover, one should not forget the boom of food delivery; indeed, Uber Eats is testing and was going to use drones for at least one portion of the food delivery process in summer 2020 in San Diego (see [3]). During the 2020 Covid-19 outbreak many drone delivery services have been considered for medical, parcel, and grocery delivery to ensure social distancing or to deliver needed goods and medications to quarantined people (see [38]). We give a few examples: Drone Delivery Canada is looking for Covid-19 use cases for drone applications (see [2]); Zipline wants to expand to the US their expertise built in Africa for blood and medical supply (see [5]); and robots have being used to deliver grocery in Washington D.C. (see [1]).

The commercial sector's interest in the use of drones justifies the interest of researcher. Recently, many studies devoted to the solution of optimization problems involving drones have been published and this article is one of those. In particular, we

study a version of the flying sidekick traveling salesman problem with multiple drones (MFSTSP). The MFSTSP considers the combined used of a truck and a set of identical drones to deliver parcels to minimize the completion time of delivery operations. Each drone can leave the truck (or the depot) to serve one customer at a time and must return to the truck. Multiple drones can take off from and return to the same place and the scheduling of these operations is included in our model.

The contribution of this article is the following:

- We propose a set of novel formulations for the MFSTSP and a comparative study, including valid inequalities adapted to solve the multiple drones case.
- We show the possible advantages of using multiple drones for delivery operations.
- We study the impact of several variants of the problem on the proposed model, focusing on the scheduling of drone activities, and we provide an extensive comparison among the variants.

In the reminder of this article, we provide a literature review in Section 2. We formally define the MFSTSP in Section 3, and in Section 4, we present four formulations. Section 5 shows the implementation details and the numerical results. Section 6 studies the impact of the scheduling of multiple drone operations on the problem. In Section 7, we study and compare other variants of the problem and Section 8 concludes this article.

2 | RELATED LITERATURE

Drones are an emerging technology and, as we have seen, their use in day-to-day operations is becoming more and more common; therefore, it is not surprising that many recent works have been focused on the use of drones. In particular, the amount of literature that considers optimization problems related to drones, and trucks and drones has increased remarkably in the last few years. A recent survey by Otto et al. [26] includes more than 300 papers that are classified into four categories: (i) Vehicles supporting operations of drones; (ii) Drones supporting operations of vehicles; (iii) Drones and vehicles performing independent tasks; (iv) Drones and vehicles as synchronized working units. The MFSTSP belongs to the last category and in particular refers to the single truck and multiple drones subcategory.

We consider, for a literature review, the category *Drones and vehicles as synchronized working units*, that includes routing problems in which trucks are equipped with drones, both vehicles can be used to deliver parcels to customers, and synchronization is important. We first examine the case with one truck and one drone and subsequently the cases that treat multiple trucks and/or drones.

2.1 | Single truck and drone

The first problem that involves the use of one truck and one drone is the *flying sidekick traveling salesman problem* (FSTSP), that was defined and solved by Murray and Chu [24]. In the FSTSP, the truck and the drone cooperate to serve all customers. The drone can leave the truck to serve one customer and returns to the truck when this is stationary. The truck and the drone must be synchronized and thus wait for each other, but no scheduling is needed, since only one drone is used. The objective function is to minimize the completion time. In the FSTSP, customers can be serviced only once, either by the truck or by the drone; however, some customers must be visited by the truck, because their request cannot be fulfilled by the drone. In this form of the problem, the drone cannot return at the launching point and each drone mission is limited by a finite endurance. In their work, Murray and Chu propose a mixed integer linear programming (MILP) formulation and three heuristic methods, based on the nearest neighbor, savings, and sweep algorithms, adapted to include the drone flights in the solution.

The *TSP with drone* (TSP-D), presented in Agatz et al. [6] for the first time, shares the main characteristics with the FSTSP. However, in the TSP-D the customers can be visited more than once by the truck if it is convenient for launching and collecting a drone; the drone can return to the same node in which it has been launched; endurance is unlimited and launching and rendezvous times are considered as negligible. The authors present an integer linear programming model and route first-cluster second heuristics. Bouman et al. [7] extend the work in Agatz et al. [6] by solving the TSP-D with dynamic programming, which is also used to define a heuristic algorithm.

Ha et al. [16] solve a FSTSP for which they propose two heuristic algorithms: a route first-cluster second and a cluster first-route second. In [15], the same authors solve a similar problem with a different objective function, made of four cost components that depend on the total distance traveled by the truck, that traveled by the drone, and the waiting time of both the truck and the drone. They present an MILP formulation based on Murray and Chu's one and two heuristics. Ponza's thesis [32] tackles the FSTSP proposing a modified MILP formulation with respect to the Murray and Chu's [24] one and solves it with a simulated annealing algorithm. Ponza's formulation does not allow the drone to wait on the ground at customers nodes to save battery, so the time spent in waiting for the truck is included in the computation of the used energy. De Freitas and Penna [10]

propose a randomized variable neighborhood descent for the FSTSP. In [11], the same authors propose a hybrid general variable neighborhood search algorithm for the FSTSP proposed by Murray and Chu [24] and the TSP-D proposed by Agatz et al. [6]. Poikonen et al. [30] propose a branch-and-bound (B&B) for a TSP-D version that considers a maximum endurance, drone flights that can start and end at the same node, and does not allow multiple visits by the truck. The authors also propose three heuristic algorithms: two derived from the B&B and the third that is a divide-and-conquer one. Yurek and Ozmutlu [44] propose an iterative algorithm based on a decomposition approach for the FSTSP. Dell'Amico et al. [12] solve two variants of the FSTSP, one where the drones can wait on the ground at customer nodes and one in which the drone can wait only if hovering. They propose two MILP formulations that improve the Murray and Chu's one, one where flights are represented with three-index variables and one in which the flights are represented with two-index variables. The authors consider an objective function that is equivalent to the Murray and Chu's one, but that provides higher lower bounds. They also propose a set of inequalities to be separated in a branch-and-cut fashion. The same authors propose improved formulations that could solve larger instances in [13], and a matheuristic for the same problem in [14].

2.2 | Single truck and multiple drones

Phan et al. [40] solve the *traveling salesman problem with multiple drones*, a problem in which one truck is equipped with multiple drones that must serve all the customers in order to minimize the transportation costs of both the truck and the drones. Each node must be visited once and the drones cannot return to their launching point. The endurance of the battery is considered, but the drones can wait for the truck on the ground and thus that time is not included in the battery consumption computation, as opposed to the MFSTSP considered in this article. The authors also impose a maximum time for which drones and truck can wait each other. They adapt the heuristic algorithm proposed by Ha et al. [15] and propose an adaptive large neighborhood search that is proven to be more efficient. They heuristically solve instances with up to 100 customers and at most three drones.

Murray and Raj [25] propose the *multiple flying sidekick traveling salesman problem* (also called MFSTPS) for parcel delivery with multiple drones, which is an extension of the FSTSP that includes multiple drones. The problem considers one truck and a heterogeneous fleet of drones, that may have different speed, capacity, service times, and flight and endurance limitations. Endurance is a function of the payload weight, of the travel distance without carrying a parcel and the travel distance with a parcel; however, the authors also consider versions with linear and fixed endurance (e.g., a maximum time or a maximum distances). The scheduling of arrivals and departures of drones is considered, together with a service time at nodes. Only a set of customers can be served by drones, also depending on the type of drone. A drone cannot return to its launching location. In the original version of the problem, the drones cannot wait on the ground at the customer nodes. The authors also study two additional versions of the problem. In the first one, the drone can leave the depot even if the truck is not there anymore. In the second one, launches and rendezvous do not need any intervention from the driver. The authors propose a MILP for the problem. They also propose a three-phase iterative heuristic: in the first phase, the customers are divided into two groups, one to be served by the truck and one by the drones, and a TSP is solved heuristically to determine the truck route. In the second phase, the drone flights to serve the remaining customers are generated. In the third phase, the timing of the activities is determined and local search procedures are used to improve the solution. The heuristic is tested on instances with up to 100 customers and four drones, while the MILP could solve to optimality 220 out of the 360 generated instances with up to eight customers. Moshref-Jaradi et al. [22] solve a similar problem minimizing the moment in which all customers are served by either a truck or a drone, while the return of the truck at the depot is not crucial. The drones can be launched from the truck simultaneously (no-scheduling) or leave the depot before the truck moves. The authors propose a MILP and an adaptive large neighborhood search metaheuristic. The exact algorithm could solve instances with up to 10 customers and three drones in about 35 000 s, on average. The metaheuristic solved instances with up to 100 customers and three drones. Seifried [37] proposes the *traveling salesman problem with multiple drones*. All the customers can be served either by the truck or by a set of identical drones. As in Murray and Raj, the drones can be launched and retrieved only at the customers or at the depot, they serve only one customer at a time, and they cannot return to their launching point. Drones are assumed to be at least as fast as the truck. In this version, the drones can wait at the nodes without consuming energy and no scheduling is considered for the launching and rendezvous operations at every node. The author presents a MILP model that solves instances with up to 10 customers and consider with up to nine drones.

Peng et al. [28] consider a two-echelon problem in which the truck starts and ends its route at the depot and stops in anchor points where the drones are launched to serve customers. The decisions to be made include the selection of the anchor points and the routing of the truck. Each drone can serve multiple customers, and if the battery finishes before all customers are served, the correspondant drone must return to the truck to recharge. The aim is to minimize the overall delivery time. The authors present a hybrid genetic algorithm for the problem. Hu et al. [18] solve a similar two-echelon problem for which they present an iterative heuristic. Poikonen and Golden [29] solve the *k-multi-visit drone routing problem* where one truck and a set of k drones can be used to serve a set of customers and minimize the completion time. Drones can perform multiple deliveries within a mission and

their endurance depends on the carried packages. Drones launches and rendezvous can happen at the depot or when the truck is stationary. The set of points where the drones can be launched and collected is different from the set of costumers. Drones can return to the launching point but if the rendezvous differs from the launching point then the truck must move directly to the rendezvous point. The authors propose a simple heuristic to solve the problem.

Salama and Srinivas [35] solve a problem with one truck equipped with identical drones to serve customers. Some customers can be visited only by the truck, while the others can be served also by the drones. The truck can stop along its route in the focal point of a cluster to launch and wait for several drones to serve the customers of that cluster. The authors present two models, in the first one, only the customer locations can be used as focal points, while in the second one, any point in the delivering area can be a focal point of a cluster. The problem optimizes jointly the clustering of the customers and the truck and drones routing. Two objective functions are considered and minimized: the total cost depending on the distance traveled by the truck and the drones and the number of used drones, and the completion of deliveries. Chang and Lee [8] consider a similar problem in which a truck equipped with several drones must visit the center of the clusters based on delivery locations. When stopped at the center of a cluster, the truck launches and waits for the drones to return before moving to the next cluster center. The authors first perform a clustering of the delivery location, then a TSP among them is solved, and afterwards a non-linear model is solved to adjust the cluster centers in order to reduce the delivery times. Moshref-Jaradi et al. [23] solve a problem in which a single truck can stop at customers to serve them or to launch drones multiple times to perform deliveries. In such a case, the drones can perform one delivery at a time and the truck can only leave when the last drone has returned. The objective function minimizes the time in which all the customers are served. The authors propose a MILP and an adaptive tabu search-simulated annealing heuristic. Karak and Abdelghany [20] present the *hybrid vehicle-drone routing problem for pickup and delivery services*, a problem that considers a single truck equipped with multiple drones able to serve more than one customer during each flight. Each node can be visited by the vehicle only once, while the drones can return in their launching node. Customers are served only by drones, and the truck is used only as drone depot. Both pickups and deliveries are considered, so as weight capacity and maximum endurance. The authors propose an MILP formulation and a savings based algorithm.

In Hu et al. [17], the authors solve a problem in which drones are used for inspection and not delivery. This means that the drones can visit multiple customers in their sorties. They propose heuristic algorithms.

2.3 | Multiple trucks and drones

Multiple trucks and drones problems have been treated and we report here a brief selection. Wang et al. [41] define the *vehicle routing problem with drones* (VRPD); Poikonen et al. [31] treat these problems theoretically, Daknama and Kraus [9] heuristically; Sacramento et al. [34] propose a MILP and a adaptive large neighborhood search for the VRPD; Schermer et al. [36] propose a formulation and a matheuristic; Kitjacharoenchai et al. [21] propose a MILP formulation and an adaptive insertion heuristic. Wang and Sheu [42] solve a VRP with drones in which each drone can serve multiple customers during one flight because the parcels are parachuted. Di Puglia Pugliese and Guerriero [33] propose and solve the *VRP with time windows in which each truck is equipped with drones*.

3 | PROBLEM DESCRIPTION

The MFSTSP is a version with multiple drones of the FSTSP originally defined by Murray and Chu [24] and thus it inherits its NP-hardness. The MFSTSP is the problem of serving a set of customers $C = \{1, \dots, c\}$ with either a truck or a drone. The truck starts its route from the depot 0, returns to the final depot $c+1$, and is equipped with a set D of identical flying drones that can be used to serve one customer at a time, in parallel with the truck. A drone service is called a *sortie*, that is defined by a launching node, a served customer, and a rendezvous node. Each drone can serve one customer at a time. All customers in C can be served by the truck, but only a subset $C' \subseteq C$ can be served by a drone with a sortie. The problem is built on a digraph $G = (N, A)$, where the set $N = \{0, 1, \dots, c+1\}$ represents all the nodes, while we define $N_0 = \{0, 1, \dots, c\}$ and $N_+ = \{1, \dots, c+1\}$. Let A be the set of all the arcs $(i,j), i \in N_0, j \in N_+, i \neq j$. Each arc (i,j) is associated with two non-negatives traveling times: τ_{ij}^T and τ_{ij}^D, that represent the time for traveling that arc by the truck and by the drones, respectively. The travel time matrices of the drones and the truck are normally different. Nodes 0 and $c+1$ represent the same physical point, the depot, and the traveling time between them is set to 0. Serving times at customers for both drone and truck are included in the travel times, while the time for preparing the drones at launch is given by σ^L and the rendezvous time is given by σ^R. No launch time is considered when the sortie starts form the depot, as in the FSTSP. The drones have a battery limit (endurance) of E time units, that constraints their use. The drones are launched with a full battery that is changed with a new one when they return to the truck. The rendezvous time σ^R contributes to battery consumption while σ^L does not, since the drone lies on the truck when it is prepared for the launch.

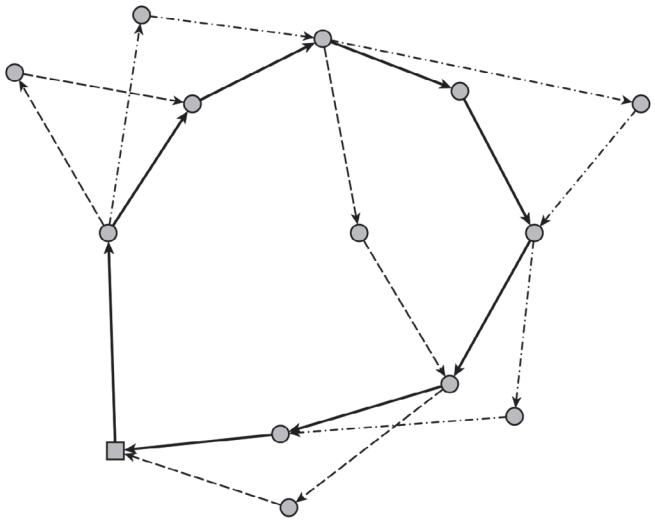


FIGURE 1 Example of a feasible solution of the MFSTSP with two drones, the dashed lines represent two different drones, while the solid arrows represent the truck path. The square is the depot and the circles are the customers

A sortie performed by a drone $v \in D$ is formally defined by a tuple (i, j, k, v), $(i \neq j, k$ and $j \neq k)$ where $i \in N_0$ is the launching node, $j \in C'$ the customer to serve, and $k \in N_+$ the rendezvous node. Let F be the set of all sorties that can be performed within the endurance time E ($\tau^D_{ij} + \tau^D_{jk} + \sigma^R \leq E$). Note that a drone v can travel a sortie $(i, j, k, v) \in F$ in at least $\tau^D_{ij} + \tau^D_{jk} + \sigma^R$, but its flight time can be larger due to the synchronization of the truck and the drones: a drone could be obliged to wait for the truck and/or for rendezvous or launching operations of other drones before being collected.

The drones can be launched from the truck only when the truck is stationary at a customer or at the depot; the drones cannot leave the depot before the truck starts its route. The truck can keep serving customers while the drones are performing their sorties; however, a synchronization is required. First of all, if a drone arrives at the rendezvous point before the truck does, then that drone must wait for the truck while hovering, and thus consuming its battery. The endurance must be large enough to allow this, otherwise that sortie is infeasible. If the truck arrives at the rendezvous point before a drone to be collected does, then the truck must wait for the drone, but it can perform other operations in the meanwhile, such as launching or collecting other drones. Indeed, the scheduling of the operations in a node is one of the main characteristics of the MFSTSP: the sequence of the operations becomes crucial. If a drone is collected and a second drone arrives in the meanwhile, the second drone must wait and this can influence its flying time and the endurance limit can be exceeded. We thus consider all the operations sequences in each node to ensure that the launching times, the rendezvous times, and the endurance limits are respected.

The objective of the optimization is to minimize the completion time, that is the moment when the last vehicle arrives at the depot. A feasible solution of a problem with two drones is depicted in Figure 1.

3.1 | Scheduling of drone operations on the truck

The scheduling component of the MFSTPS is crucial and we believe it deserves a deeper insight that we provide here, by making use of an example. In Figure 2, one can find an example of multiple operations occurring in a node, let us say $i \in C$. In Figure 2A, two drones, $D1$ and $D2$, arrive at the rendezvous point before the truck does and must wait for it. In Figure 2B, the truck arrives and in Figure 2C one of the two drones, $D1$ in the example, lands on the truck while the other must wait hovering. In Figure 2D, the truck is free and thus the second drone, $D2$, can perform the rendezvous operation. In Figure 2E, drone $D1$ is prepared to be launched to perform a new sortie and in the meantime drone $D3$ arrives and thus it must wait for the truck to be free. When the launching operation of drone $D1$ ends, drone $D3$ can land on the truck as in Figure 2F. The drone $D2$ can now perform a new sortie (see Figure 2G), and the truck can now move to the following node on its route as in Figure 2H.

In Figure 3, the Gantt chart of this example is depicted. In green we show the flights of the drones when moving from one node to another, in gray the time the drones wait while hovering, in red the truck route, and in blue and yellow the rendezvous and launch operations, respectively. No color is used when the drones are hosted on the truck. In the chart of the example, drone $D1$ arrives first at time A_{D1} and then $D2$ arrives at A_{D2}. They must wait for the truck that only arrives at A_T. $D1$ and $D2$ are then collected and $D1$ is launched, so it departs from the truck at P_{D1}. Note that these operations cannot be performed in parallel. Moreover, a drone can be launched only if it is present on the truck and thus if the rendezvous and the launch of a drone occur at the same node, they must be in this order. Meanwhile $D3$ arrives (A_{D3}), waits for the truck to be free, and then is collected and remains on the truck. $D2$ is thus launched so to leave the truck at P_{D2}, which coincides to P_T, the time in which the truck

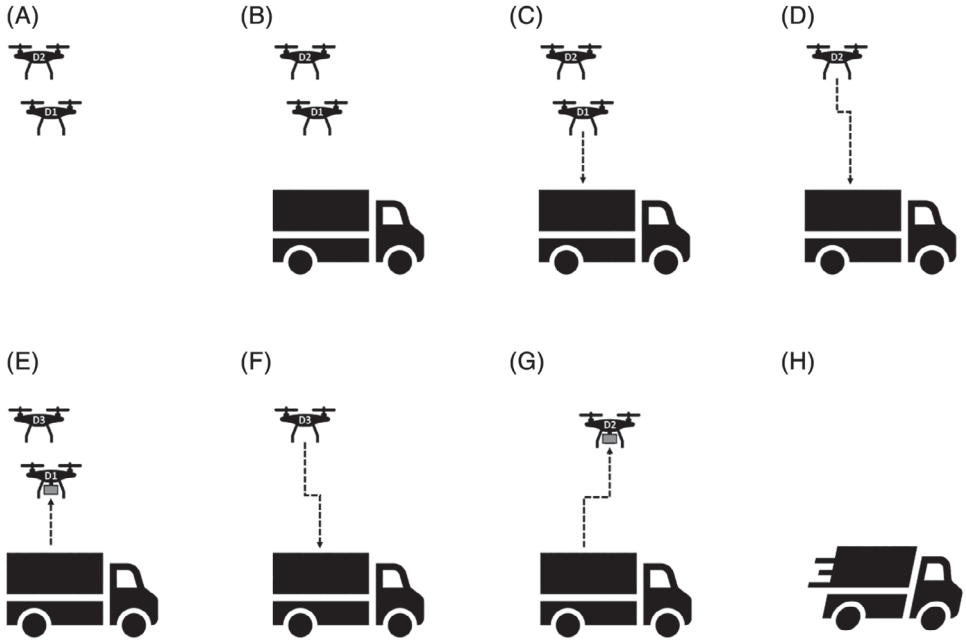


FIGURE 2 Sequence of operations of multiple drones and the truck that can occur at a given node

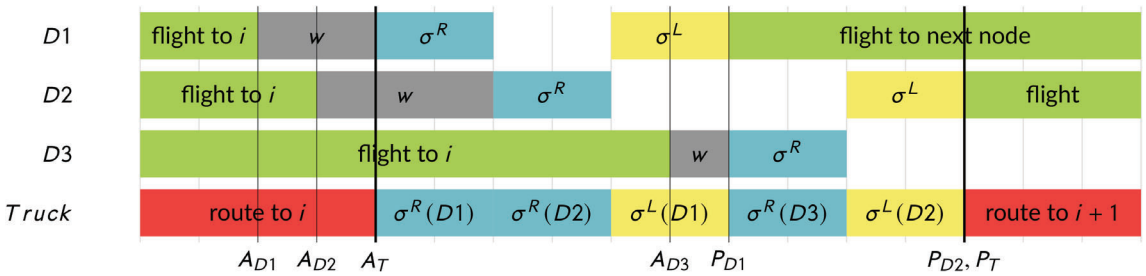


FIGURE 3 Example of a schedule of multiple operations at a given node i [Color figure can be viewed at wileyonlinelibrary.com]

can start and proceed to node $i+1$, the following node on its route. Observe that if the only operation to be performed at node i were to collect drone $D3$, the truck should have waited to perform the rendezvous operations of $D3$ from A_T to A_{D3}.

The previous example showed that in each node visited by the truck, the truck can be considered as a single machine of a scheduling problem on which a set of drone operations take place: these operations are the launch and rendezvous of drones. The truck is a single machine because each of these operations can be done one at a time due to the truck structure. Moreover, the starting times of rendezvous and launch operations are bounded by what we can call the earliest and the latest rendezvous time and the earliest and the latest time of departure of each drone, respectively. Let us consider that a drone v is returning on the truck at node i after a sortie (k, j, i, v). The earliest rendezvous time of that drone is the maximum value between the arrival time of the truck at node i (t_i^P) and the time in which the drone is ready for rendezvous at node i, given by the departure time of the drone in k and the flight time $\tau_{kj}^D + \tau_{ji}^D$. The latest rendezvous time of the same sortie is linked to the battery endurance and is given by the time of departure of drone v from node k plus $E - \sigma^R$. Note that σ^R is included into the flying time but is not included into the travel time in this representation, since we see it as a jobs to be done on a machine (the truck). Let us now consider a drone w launched from the truck at node i: if the drone were already on the truck then the earliest launch time would be the arrival of the truck at node i, otherwise it would be the starting time of rendezvous plus σ^R (which is linked to the rendezvous operation decision). The latest launch time is minimized by the overall objective function. Due to the considerations upon the earliest and latest times, we can conclude that all the decisions on the times of operations in each node depend also on decisions made in other nodes of the truck route, if one wants to avoid infeasibilities.

We see in Figure 4 that if $D1$ is scheduled before $D2$, then the truck can leave node i in advance, which goes in the direction of the objective function; however, such a scheduling solution is infeasible because drone $D2$ cannot respect the latest rendezvous time and thus it exceeds its battery endurance. Scheduling drone $D2$ first and drone $D1$ second, instead, increases the costs but leads to a feasible solution. Observe that the same reasoning can be extended to cases in which launch and rendezvous operations of more drones are involved at the same node.

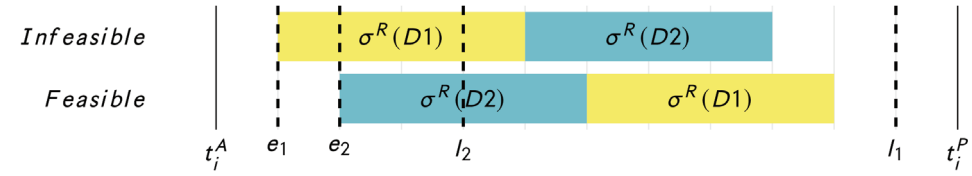


FIGURE 4 An example of feasible and infeasible drone operations schedules at a node i between the truck arrival and departure times t_i^A and t_i^P, in which drones *D1* and *D2* must start their rendezvous operations respecting the earliest (e_1 and e_2) and latest (l_1 and l_2) rendezvous times [Color figure can be viewed at wileyonlinelibrary.com]

4 | MATHEMATICAL FORMULATIONS

4.1 | Four-index formulation

In this section, we propose a formulation that we call *4I-BC* because it uses four-index variables to represent the sorties, and because some constraints are separated in a branch-and-cut fashion. 4I-BC makes use of a first set of binary variables x_{ij} that equal 1 if the arc $(i,j) \in A$ is traveled by the truck, 0 otherwise. We recall that the set of arcs A includes only arcs (i,j) such that $i \in N_0$, $j \in N_+$, and $i \neq j$. The second set of variables used by 4I-BC models the sorties: y_{ijk}^v equals 1 if the sortie $(i,j,k,v) \in F$ is performed, 0 otherwise. To impose the synchronization, a set of time variables is needed. First of all we introduce the truck arrival and departure time variables t_i^A, t_i^P for each node $i \in N$. Moreover, we introduce the drone arrival and departure times t_i^{vA}, t_i^{vP} for each node $i \in N$ and drone $v \in D$. To handle the scheduling of operations at each node, we introduce the following binary variables: λ_i^{vw}, that equals 1 if drone $v \in D$ is launched from node $i \in N_0$ before drone $w \in D, w \neq v$ is, 0 if this is not the case; ρ_i^{vw}, that equals 1 if drone $v \in D$ returns to the truck at node $i \in N_+$ before drone $w \in D, w \neq v$ does, 0 if this is not the case; α_i^{vw}, that equals 1 if drone $v \in D$ is launched from $i \in N$ before $w \in D, w \neq v$ lands on the truck at node $i \in N$, 0 otherwise; and β_i^{vw}, that equals 1 if drone $v \in D$ returns to the truck at node $i \in N$ before drone $w \in D, w \neq v$ is launched from $i \in N$, 0 otherwise. The resulting formulation is described step by step in the following.

4.1.1 | Objective function

According to the problem description, the objective function aims at minimizing the completion time, which is the latest arrival time at the final depot with respect to all the vehicles (truck and drones). This can be expressed as the minimization of the truck departure time at the final depot: $\min t_{c+1}^P$, given that it must be greater than or equal to all the arrival times; however, Dell'Amico et al. [12] showed that this objective function provides noneffective lower bounds for the single drone problem (the FSTSP) and that a decomposition is more effective. We thus decompose the objective function into an equivalent one: the first component is given by the traveling times of the truck route, indeed we need to wait at least the truck route time before returning to the depot. The second component is given by the difference between the truck departure and arrival times for each node. We will show in the following that this difference includes the launching and rendezvous times and the waiting times. The second component can be written as $\sum_{i \in N}(t_i^P - t_i^A)$. However, this difference can lead to bad lower bounds and thus we define a set of non-negative variables w_i, one for each node $i \in N$ to represent each of these differences. This rationale is expressed by the objective function (1) and the constraint (2).

$$\min z = \sum_{(i,j) \in A} \tau_{ij}^T x_{ij} + \sum_{i \in N} w_i \tag{1}$$

$$w_i \geq (t_i^P - t_i^A) \qquad i \in N. \tag{2}$$

4.1.2 | Routing and sortie constraints

In the following, we detail the constraints that model the truck routing and the drone flights.

$$\sum_{i:(i,j) \in A} x_{ij} + \sum_{i,k,v:(i,j,k,v) \in F} y_{ijk}^v = 1 \qquad j \in C \tag{3}$$

$$\sum_{i:(j,i) \in A} x_{ji} + \sum_{i,k,v:(i,j,k,v) \in F} y_{ijk}^v = 1 \qquad j \in C \tag{4}$$

$$\sum_{j \in N_+} x_{0j} = \sum_{i \in N_0} x_{i,c+1} = 1 \tag{5}$$

$$\sum_{i:(i,j) \in A} x_{ij} = \sum_{k:(j,k) \in A} x_{jk} \qquad j \in C \tag{6}$$

$$\sum_{j,k:(i,j,k,v)\in F} y^v_{ijk} \le 1 \qquad i \in N_0, v \in D \tag{7}$$

$$\sum_{i,j:(i,j,k,v)\in F} y^v_{ijk} \le 1 \qquad k \in N_+, v \in D \tag{8}$$

$$2y^v_{ijk} \le \sum_{h\in N_0, h\neq k} x_{hi} + \sum_{l\in C, l\neq k} x_{lk} \qquad i,j,k,v : (i,j,k,v) \in F \tag{9}$$

$$y^v_{0jk} \le \sum_{h\in N_0, h\neq k} x_{hk} \qquad j,k,v : (0,j,k,v) \in F. \tag{10}$$

Constraints (3) and (4) impose that each customer is visited exactly once by the truck or one of the drones, if the drone visit is possible. Note that the linear combination of these constraint also imposes the flow conservation constraint (6). Constraint (5) guarantees that the truck starts its route form the starting depot 0 and returns to the ending depot. Note that if $x_{0,c+1} = 1$ only drones are used to serve customers, but each drone can be used only once in such a case. Constraints (7) and (8) guarantee that a drone is launched and collected at most once in a certain node. Constraint (9) guarantees that a sortie is performed only if its launching and rendezvous nodes are visited by the truck. Constraint (10) imposes the same concept for sorties starting from the depot. One should note that a typical subtour elimination constraint is missing, this is because subtours are eliminated thanks to the timing constraints; however, we can easily avoid two nodes subtours by imposing $x_{ij} + x_{ji} \le 1, i,j \in N, i \neq j$.

4.1.3 | Truck timing and drone timing

To impose the synchronization of operations, we include the following constraints.

$$t^A_j \ge t^P_i + \tau^T_{ij} - M(1 - x_{ij}) \qquad (i,j) \in A \tag{11}$$

$$t^A_j \le t^P_i + \tau^T_{ij} + M(1 - x_{ij}) \qquad (i,j) \in A \tag{12}$$

$$t^{vA}_k \ge t^{vP}_i + \tau^D_{ij} + \tau^D_{jk} + \sigma^R - M(1 - y^v_{ijk}) \qquad (i,j,k,v) \in F \tag{13}$$

$$t^{vA}_k - t^{vP}_i \le E + M\left(1 - \sum_{j\in C'} y^v_{ijk}\right) \qquad i \in N_0, k \in N_+, v \in D. \tag{14}$$

Constraints (11) and (12) guarantee that if an arc (i,j) is traveled by the truck, then the arrival time of the truck in j is the departure time of the truck in i plus the time to travel the arc. The combination of constraints (11) and (12) imposes the equality $t^A_j = t^P_i + \tau^T_{ij}$, if the arc (i,j) is traveled. The equality guarantees that the waiting times are included between the truck departure and arrival times at every node and thus that the objective function decomposition holds. If drone $v \in D$ performs the sortie $(i,j,k,v) \in F$, then the arrival time in k should include the departure time in i, the traveling times τ^D_{ij} and τ^D_{jk}, and the rendezvous time: this is guaranteed by constraint (13). We recall that the launching time is not considered in the flying time and thus it must be considered before t^{vA}_i. If the sortie is performed, then the difference between the arrival time and the departure time of the drone must not exceed the endurance E as imposed by constraint (14).

4.1.4 | Timing in a vertex

Hereby we model what occurs in a node between the arrival and the departure of the truck. Constraint (15) imposes that the arrival time of a drone at a node follows the arrival time of the truck and if a sortie ends in that node then the rendezvous time must also be included. Note that if the drone finishes its sortie before the truck arrives at the meeting point, the drone arrival will be extended to include this (if allowed by the endurance constraint). In terms of costs in the objective function, the drones waiting time is absorbed by the truck time. In case the truck waits for one or more drones, the waiting time is included between the truck arrival and departure times of the node in which the truck is waiting. Constraint (16) states that the departure time of the drones at node 0 is greater than or equal to the arrival time; in constraint (17) it is also imposed that the drone departure time must be greater than or equal to the arrival time of the drone at the same node plus the launching time, if the drone is launched from that node. Constraint (18) guarantees that the truck leaves one node only when the handling of all the drones is completed.

$$t^{vA}_i \ge t^A_i + \sigma^R \sum_{k,j:(k,j,i,v)\in F} y^v_{kji} \qquad i \in N, v \in D \tag{15}$$

$$t^{vP}_0 \ge t^{vA}_0 \qquad v \in D \tag{16}$$

$$t_i^{vP} \geq t_i^{vA} + \sigma^L \sum_{j,k:(i,j,k,v)\in F} y_{ijk}^v \qquad i \in N_+, v \in D \tag{17}$$

$$t_i^P \geq t_i^{vP} \qquad i \in N, v \in D. \tag{18}$$

4.1.5 | Scheduling in a vertex

The following constraints are needed to impose that the arrival and departure times of the drones respect the sequence of launches and rendezvous, that is, the scheduling of operations in a node. Constraint (19) imposes the following: if drone w is launched before drone v is from node i, then the departure time of drone v must be at least the departure time of drone w plus the launching time (not included in the flight time). Note that the launching time is not included if the drone is launched from the starting depot. Constraint (20) states that, if the drones w and v are both collected at node i and drone w returns before drone v does, then the time of arrival of drone v must be at least the time of arrival of drone w plus the rendezvous time (which is included in the flying time). If drone w returns to i before the drone v is launched, then the departure time of v must be at least the arrival time of w plus the launching time. If the drone w is launched from i before drone v is returned, then the arrival time of v must be greater than or equal to the departure time of w plus the rendezvous time. These two conditions are expressed by constraints (21) and (22), respectively.

$$t_i^{vP} \geq t_i^{wP} + \sigma^L - M(1 - \lambda_i^{wv}) \qquad i \in N_+, v, w \in D, v \neq w \tag{19}$$

$$t_i^{vA} \geq t_i^{wA} + \sigma^R - M(1 - \rho_i^{wv}) \qquad i \in N_+, v, w \in D, v \neq w \tag{20}$$

$$t_i^{vP} \geq t_i^{wA} + \sigma^L - M(1 - \beta_i^{wv}) \qquad i \in N_+, v, w \in D, v \neq w \tag{21}$$

$$t_i^{vA} \geq t_i^{wP} + \sigma^R - M(1 - \alpha_i^{wv}) \qquad i \in N_+, v, w \in D, v \neq w. \tag{22}$$

4.1.6 | Linking constraints

The following constraints are needed to link the scheduling variables to the sortie variables.

$$\rho_k^{vw} + \rho_k^{wv} \leq 1/2 \sum_{i\in N_0} \sum_{j\in C'} (y_{ijk}^v + y_{ijk}^w) \qquad k \in N_+, w, v \in D, w \neq v \tag{23}$$

$$\rho_k^{vw} + \rho_k^{wv} + 1 \geq \sum_{i\in N_0} \sum_{j\in C'} (y_{ijk}^v + y_{ijk}^w) \qquad k \in N_+, w, v \in D, w \neq v \tag{24}$$

$$\lambda_i^{vw} + \lambda_i^{wv} \leq 1/2 \sum_{j\in C'} \sum_{k\in N_+} (y_{ijk}^v + y_{ijk}^w) \qquad i \in N_0, w, v \in D, w \neq v \tag{25}$$

$$\lambda_i^{vw} + \lambda_i^{wv} + 1 \geq \sum_{j\in C'} \sum_{k\in N_+} (y_{ijk}^v + y_{ijk}^w) \qquad i \in N_0, w, v \in D, w \neq v \tag{26}$$

$$\alpha_i^{vw} + \alpha_i^{wv} \leq 1 \qquad i \in N, v, w \in D, v \neq w \tag{27}$$

$$\alpha_i^{vw} + \beta_i^{wv} \leq 1/2 \left(\sum_{j\in C'} \sum_{k\in N_+} y_{ijk}^v + \sum_{k\in N_0} \sum_{j\in C'} y_{kji}^w \right) \qquad i \in N, v, w \in D, v \neq w \tag{28}$$

$$\alpha_i^{vw} + \beta_i^{wv} + 1 \geq \left(\sum_{j\in C'} \sum_{k\in N_+} y_{ijk}^v + \sum_{k\in N_0} \sum_{j\in C'} y_{kji}^w \right) \qquad i \in N, v, w \in D, v \neq w. \tag{29}$$

To explain constraint (23), we recall that ρ_k^{vw} equals one if two drones, v and w, return to the same node k and drone v returns before w. In that constraint, we impose that at most one between ρ_k^{vw} and ρ_k^{wv} can take value one and this can happen only when two sorties, one for v and one for w, return in k. Constraint (24) completes the reasoning by imposing that one drone must return before the other if two drones return to the same node. Constraints (25) and (26) ensure the same for launches represented by variables λ. To explain the remaining constraints, we recall that β_i^{vw} equals one if drone v returns to i before w is launched from the same node, and that α_i^{vw} equals one if drone v is launched from node i before drone w returns to the same node. Note that both β_i^{vw} and β_i^{wv} can both take value one, for instance when this sequence happens: w returns, v returns, v departs, and w departs. In constraint (27), we impose that either v is launched before w returns to i, or the other way round, if sorties involving drones v

and w occur at that node. In constraints (28) and (29), we impose that, if drone v is returning to i and drone w is launched from the same node, then one operation takes place before the other.

4.1.7 | Crossing sortie elimination

The formulation we have described up to now could lead to infeasible solutions because it allows the so-called *crossing sorties*, defined properly as follows, to happen. Let $i \in N_0, l \in C$ be two nodes visited by the truck while running along a path $P(v)$ from i to l, and assume that two sorties of the same drone $v \in D$ (i,j,k,v) and (l,m,n,v) with $k \notin P(v)$ exist. In this case, the second sortie starts before the first one finishes and the overall solution is therefore infeasible. Let us define $\mathcal{P}(v)$ as the set of all the paths with these characteristics for each drone $v \in D$. The following *crossing sorties elimination constraints* (30), adapted from those proposed by Dell'Amico et al. [12] to account for multiple drones, can be used to eliminate this type of infeasibility.

$$\sum_{h=1}^{|P(v)|-1} x_{s(h),s(h+1)} + \sum_{\substack{j,k:(i,j,k,v)\in F \\ k\notin P(v)}} y^v_{ijk} + \sum_{l,m:(l,m,n,v)\in F} y^v_{lmn} \le |P(v)| \qquad P(v) \in \mathcal{P}(v), v \in D \tag{30}$$

where $P(v) = \{s(1), s(2), \ldots, s(|P(v)|)\}$, $s(1) = i$, $s(|P(v)|) = l$. These constraints impose that at least one of the arcs of the path or one of the variables representing the sorties is set to zero to make the solution feasible.

We can strengthen (30) using the idea of "tournament constraint". Since the path enters at most once in each nodes, we can include in the constraint also the arcs $(h,j) \in A$ such that $h, j \in P(v)$ and h precedes j in $P(v)$. We obtain the *tournament crossing constraints* (TCS) given by (31), where $P(v) = \{s(1), s(2), \ldots, s(|P|(V)|\}$.

$$\sum_{h=1}^{|P(v)|-1} \sum_{j=h+1}^{|P(v)|} x_{s(h)s(j)} + \sum_{\substack{j,k:(s(1),j,k,v)\in F \\ k\notin P(v)}} y^v_{s(1)jk} + \sum_{m,n:(s(|P(v)|),m,n,v)\in F} y^v_{s(|P(v)|)mn} \le |P(v)| \qquad P(v) \in \mathcal{P}(v), v \in D. \tag{31}$$

4.1.8 | Backward sortie elimination

Let $\mathcal{B}(v)$ denote the set of all truck paths $P(v) = \{s(1), s(2), \ldots, s(q)\}$ with $s(1) = 0$ and $s(q) \in C$ such that it exists a sortie $(i,j,s(q),v)$ with $i \notin P(v)$, $v \in D$. The *backward sortie elimination constraints* that account for multiple drones are given by (32), and impose that at least one of the arcs of the path or the sortie is eliminated.

$$\sum_{h=1}^{|P(v)|-1} x_{s(h)s(h+1)} + \sum_{j:(i,j,s(q),v)\in F} y^v_{ijs(q)} \le |P(v)| - 1 \qquad P(v) \in \mathcal{B}(v), v \in D. \tag{32}$$

Constraint (32) can be strengthened by considering a tournament type constraint on path $P(v)$, that imposes that at most one arc enters a node of the path, and adding all sorties of drone $v \in D$ that terminate in $P(v)$, but start at nodes outside $P(v)$. We obtain the *tournament backward constraints* (TBS), given by (33).

$$\sum_{h=1}^{|P(v)|-1} \sum_{j=h+1}^{|P(v)|} x_{s(h)s(j)} + \sum_{\substack{i,j,k:(i,j,k,v)\in F \\ i\notin P(v),\ k\in P(v)}} y^v_{ijk} \le |P(v)| - 1 \qquad P(v) \in \mathcal{B}(v), v \in D. \tag{33}$$

Finally, note that backward sorties happen only when the time constraints are not respected. In our case, in fractional solutions.

4.1.9 | Bounds

$$t^A_0 = t^P_0 = 0 \tag{34}$$

$$t^A_i, t^P_i \ge 0 \qquad i \in N_+, v \in D \tag{35}$$

$$t^{vA}_i, t^{vP}_i \ge 0 \qquad i \in N, v \in D \tag{36}$$

$$x_{ij} \in \{0,1\} \qquad (i,j) \in A \tag{37}$$

$$y^v_{ijk} \in \{0,1\} \qquad (i,j,k,v) \in F \tag{38}$$

$$w_i \geq 0 \qquad i \in N \tag{39}$$

$$\alpha_i^{vw}, \beta_i^{vw}, \rho_i^{vw}, \lambda_i^{vw} \in \{0,1\} \qquad i \in N, v, w \in D, v \neq w. \tag{40}$$

4.2 | Three-index formulation

The second formulation we present, called *3I-BC*, uses a smaller number of variables similar to what was done by Dell'Amico et al. [12]. In particular, we treat each sortie with two sets of three-index variables, instead of one four-index set: the binary variable $\overrightarrow{g}_{ij}^{v}$ that equals 1 if a sortie of drone $v \in D$ starts at node $i \in N_0$ and serves node $i \in C'$, 0 otherwise; and the binary variable $\overleftarrow{g}_{ij}^{v}$ that equals 1 if a sortie of drone $v \in D$ returns to node $j \in N_+$ after serving node $i \in C'$, 0 otherwise. The other variables remain the same with respect to the previous formulation.

To further reduce the number of variables, we preliminary fix to zero all the variables corresponding to arcs with flying time exceeding the battery limit, that is, we set $\overrightarrow{g}_{ij}^{v} = 0$ for all $(i,j) \in A : \tau_{ij}^{D} > E$ and $\overleftarrow{g}_{jk}^{v} = 0$ for all $(j,k) \in A : \tau_{jk}^{D} + \sigma_R > E$, $v \in D$. We also fix to zero all those variables that do not allow to complete a feasible drone flight: $\overrightarrow{g}_{ij}^{v} = 0$, $(i,j) \in A, j \notin C'$; $\overleftarrow{g}_{jk}^{v} = 0$, $(j,k) \in A, j \notin C'$; $\overrightarrow{g}_{ic+1}^{v} = 0$, $i \in N$; and $\overleftarrow{g}_{j0}^{v} = 0$, $j \in N$, $v \in D$.

Formulation 3I-BC inherits some components from 4I-BC: first of all the objective function (1) and the constraint on variables w (2); the routing constraints (5) and (6); the timing constraints (11), (12), (16), (18)–(22); the linking constraint (27); and the variable bounds (34)–(37), (39), and (40).

4.2.1 | Routing and sortie constraints

Similar to what was done for formulation 4I-BC, we need to express the routing and sortie constraints with the new variables.

$$\sum_{i:(i,j)\in A} x_{ij} + \sum_{i:(i,j)\in A} \overrightarrow{g}_{ij}^{v} = 1 \qquad j \in C, v \in D \tag{41}$$

$$\sum_{j:(i,j)\in A} x_{ij} + \sum_{j:(i,j)\in A} \overleftarrow{g}_{ij}^{v} = 1 \qquad i \in C, v \in D \tag{42}$$

$$\sum_{j:(i,j)\in A} \overrightarrow{g}_{ij}^{v} \leq \sum_{h:(i,h)\in A} x_{ih} \qquad i \in N_0, v \in D \tag{43}$$

$$\sum_{i:(i,j)\in A} \overleftarrow{g}_{ij}^{v} \leq \sum_{h:(h,j)\in A} x_{hj} \qquad j \in N_+, v \in D \tag{44}$$

$$\sum_{i:(i,j)\in A} \overrightarrow{g}_{ij}^{v} = \sum_{k:(j,k)\in A} \overleftarrow{g}_{jk}^{v} \qquad j \in C', v \in D. \tag{45}$$

Constraints (41) and (42) are needed to impose that each customer is served either by the truck or by a drone. Constraints (43) and (44) impose that a sortie starts from and returns to nodes served by the truck; while constraint (45) is needed to impose the flow conservation at every node served by drones.

4.2.2 | Truck timing and drone timing

Also the truck and drone timing constraints need to be updated to embrace the new variables.

$$t_k^{vA} \geq t_i^{vP} + \tau_{ij}^{D} + \tau_{jk}^{D} + \sigma^R - M(2 - \overrightarrow{g}_{ij}^{v} - \overleftarrow{g}_{jk}^{v}) \qquad i \in N_0, j \in C', k \in N_+, i \neq j, k,\ j \neq k, v \in D \tag{46}$$

$$t_k^{vA} - t_i^{vP} \leq E + M(2 - \overrightarrow{g}_{ij}^{v} - \overleftarrow{g}_{jk}^{v}) \qquad i \in N_0, j \in C', k \in N_+, v \in D. \tag{47}$$

Constraint (46) states that if a sortie is performed then the arrival time of the sortie at the rendezvous node is greater than or equal to its departure time plus the traveling and the rendezvous times. Constraint (47) guarantees that the time needed to perform a sortie does not exceed the battery endurance.

4.2.3 | Timing in a vertex

We need to consider the timing in a vertex and update the related constraints.

$$t_i^{vA} \geq t_i^{A} + \sigma^R \sum_{j \in C'} \overleftarrow{g}_{ji}^{v} \qquad i \in N, v \in D \tag{48}$$

$$t_i^{vP} \geq t_i^{vA} + \sigma^L \sum_{j \in C'} \overrightarrow{g}_{ij}^{v} \qquad i \in N_+, v \in D. \tag{49}$$

Constraint (48) imposes that the arrival time of a drone includes the rendezvous time. Constraint (49) includes the launching time in a node if a sortie is launched there.

4.2.4 | Linking constraints

As in the previous formulation, we need to represent the sequence of arrivals and departures of drones and thus we need to link the needed variables with those defining the sorties.

$$\rho_k^{vw} + \rho_k^{wv} \leq 1/2 \sum_{j \in C'} (\overleftarrow{g}_{jk}^{v} + \overleftarrow{g}_{jk}^{w}) \qquad k \in N_+, w, v \in D, w \neq v \tag{50}$$

$$\rho_k^{vw} + \rho_k^{wv} + 1 \geq \sum_{j \in C'} (\overleftarrow{g}_{jk}^{v} + \overleftarrow{g}_{jk}^{w}) \qquad k \in N_+, w, v \in D, w \neq v \tag{51}$$

$$\lambda_i^{vw} + \lambda_i^{wv} \leq 1/2 \sum_{j \in C'} (\overrightarrow{g}_{ij}^{v} + \overrightarrow{g}_{ij}^{w}) \qquad i \in N_0, w, v \in D, w \neq v \tag{52}$$

$$\lambda_i^{vw} + \lambda_i^{wv} + 1 \geq \sum_{j \in C'} (\overrightarrow{g}_{ij}^{v} + \overrightarrow{g}_{ij}^{w}) \qquad i \in N_0, w, v \in D, w \neq v \tag{53}$$

$$\alpha_i^{vw} + \beta_i^{wv} \leq 1/2 \left(\sum_{j \in C'} \overrightarrow{g}_{ij}^{v} + \sum_{j \in C'} \overleftarrow{g}_{ji}^{w} \right) \qquad i \in N, v, w \in D, v \neq w \tag{54}$$

$$\alpha_i^{vw} + \beta_i^{wv} + 1 \geq \left(\sum_{j \in C'} \overrightarrow{g}_{ij}^{v} + \sum_{j \in C'} \overleftarrow{g}_{ji}^{w} \right) \qquad i \in N, v, w \in D, v \neq w. \tag{55}$$

Constraints (50) and (51) impose that if two sorties are returning to the same node, then one arrives before the other. Constraints (52) and (53) impose that if two sorties are launched from the same node, one is launched before the other. Constraints (54) and (55) impose that, if one sortie is returning to one node and another sortie is leaving, then one action must happen before the other.

4.2.5 | Crossing sortie elimination

TCS (31) must be rewritten as follows. Let $i \in N_0, l \in C$ be two nodes encountered by the truck while running along a path $P(v)$ from i to l, and assume that exist two sorties launches of drone $v \in D$ defined by $\overrightarrow{g}_{ij}^{v} > 0$ and $\overrightarrow{g}_{lm}^{v} > 0$ and such that there is no node $k \in P(v) \setminus \{i\}$ with $\overleftarrow{g}_{jk}^{v} > 0$. In this case, the second sortie starts before the first sortie is terminated an the following tournament crossing constraint (TCS2) holds:

$$\sum_{h=1}^{|P(v)|-1} \sum_{j=h+1}^{|P(v)|} x_{s(h)s(j)} + \sum_{\substack{j:(i,j) \in A, \\ j \notin P(v)}} \overrightarrow{g}_{ij}^{v} + \sum_{\substack{m:(l,j) \in A, \\ m \notin P(v)}} \overrightarrow{g}_{lm}^{v} \leq |P(v)| \qquad P(v) \in \mathcal{P}(v), v \in D \tag{56}$$

where $P(v) = \{s(1), s(2), \ldots, s(|P(v)|)\}$ with $s(1) = i, s(|P(v)|) = l$, and $\mathcal{P}$(v) now defines the set of all the paths with the described characteristics with respect to drone $v \in D$.

4.2.6 | Backward sorties elimination

Backward sorties elimination constraints are modified as follows. Let $i \in N_0, j \in N_+$ be two nodes encountered by the truck while traveling on a path $P(v)$ and i is encountered before j. Suppose that a sortie rendezvous identified by $\overleftarrow{g}_{ki}^{v} > 0$ and such that $k \notin P$, and a sortie launch identified by $\overrightarrow{g}_{jk}^{v} > 0$ and such that $k \notin P(v)$ both exist and that together they determine an infeasible solution. Let $\mathcal{B}(v)$ now denote the set of all truck paths $\{s(1), \ldots, s(m), \ldots, s(q)\}$ with $s(1) = 0$, $s(m) = i$, and $s(q) = j$ with such a sortie as described for drone $v \in D$. The tournament version of the backward sorties elimination constraints (TBS2) is:

$$\sum_{h=1}^{|P(v)|-1} \sum_{l=h<+1}^{|P(v)|} x_{s(h)s(l)} + \overrightarrow{g}_{s(|P(v)|),k}^{v} + \sum_{\substack{l:(l,k) \in A \\ l \notin P(v)}} \overrightarrow{g}_{lk}^{v} + \sum_{\substack{l:(k,l) \in A \\ l \in P(v)}} \overleftarrow{g}_{kl}^{v} \leq |P(v)| \qquad P(v) \in \mathcal{B}(v), v \in D, k \in C', k \notin P(v). \tag{57}$$

4.2.7 | Bounds and other constraints

We finally need to impose the following constraints to avoid infeasibilities:

$$\overrightarrow{g}^v_{ij} + \overleftarrow{g}^v_{ij} \leq 1 \qquad (i,j) \in A, v \in D \tag{58}$$

$$\overrightarrow{g}^v_{ij} + \overleftarrow{g}^v_{ji} \leq 1 \qquad (i,j) \in A, v \in D \tag{59}$$

$$\overrightarrow{g}^v_{ij}, \overleftarrow{g}^v_{ij} \in \{0,1\} \qquad (i,j) \in A, v \in D. \tag{60}$$

4.3 | Crossing sortie variables formulation with four-index variables

We define a new formulation based on 4I-BC that we call *4I-z*. Mainly, we replicate most of the four-index constraints and the objective function, in particular (1)–(29) and (34)–(40). Along the lines of what was done in Dell'Amico et al. [13], we introduce the *crossing sortie variables*: new binary variables z^v_i, one for each node $i \in N$ and each drone $v \in D$ that take value 1 if the drone v is available for launching at node i, 0 otherwise. To guarantee this, we introduce the following constraints:

$$\sum_{j,k:(i,j,k,v)\in F} y^v_{ijk} \leq z^v_i \qquad i \in N_0, v \in D \tag{61}$$

$$z^v_i \leq \sum_{j:(j,i)\in A} x_{ji} \qquad i \in N_+, v \in D \tag{62}$$

$$z^v_j \leq z^v_i - x_{ij} + \sum_{l,k:(l,k,j,v)\in F} y^v_{lkj} - \sum_{k,l:(i,k,l,v)\in F} y^v_{ikl} + 1 \qquad (i,j) \in A, v \in D. \tag{63}$$

$$z^v_i \in \{0,1\} \qquad i \in N, v \in D. \tag{64}$$

Constraint (61) guarantees that variable z^v_i equals 1 if a sortie of drone $v \in D$ starts at node $i \in N_0$. Constraint (62) guarantees that a variable z^v_i assumes value 1 only if the truck in entering in node $i \in N_+$. Constraint (63) allows the z to be 1 until a sortie starts, after that point the z variable is enforced to be 0 on the truck path until a sortie of the same drone returns to the truck, when the variable z is allowed to assume value 1 once again. This constraint imposes that the variable z is 0 when the corresponding drone is not on the truck. In (64), the variables z^v_i are defined as binary.

Note that no sortie elimination constraint is needed because the new variables guarantee the avoidance of crossing sorties. However, one could include TCS (31) and also TBS (33) to strengthen this formulation by cutting infeasible fractional solutions.

4.4 | Crossing sortie variables formulation with three-index variables

We combine the idea of the crossing sortie variables to the three-index variables to model the sorties, obtaining a new formulation called *3I-z*. In particular, we use the following components: objective function (1) and constraint (2); the routing constraints (5) and (6); the timing constraints (11), (12), (16), (18)–(22); the linking constraint (27); and the variable bounds: (34)–(37), (39)–(55), and (58)–(60). The variables z^v_i are the same as in the 4I-z, and hence we need to include the linking constraint (62) and the variable bound (64) in 3I-z. We thus add constraints (65) and (66) that work alike constraints (61) and (63), respectively:

$$\sum_{j\in C'} \overrightarrow{g}^v_{ij} \leq z^v_i \qquad i \in N_0, v \in D \tag{65}$$

$$z^v_j \leq z^v_i - x_{ij} + \sum_{k\in C'} (\overleftarrow{g}^v_{kj} - \overrightarrow{g}^v_{ik}) + 1 \qquad (i,j) \in A, v \in D. \tag{66}$$

Note that, also for 3I-z, no sortie elimination constraint is needed since the new variables guarantee the avoidance of crossing sorties. However, we can use TCS2 (56) and also TBS2 (57) to strengthen this formulation by cutting infeasible fractional solutions.

5 | COMPUTATIONAL EXPERIMENTS

We have implemented the proposed methods in C++ and run them on a computer equipped with an Intel Core 3.10 GHz i3-2100 CPU and with 8.00 GB of RAM, running Windows 7 operating system. CPLEX 12.71 was used as MILP solver, and

only a single thread was utilized during the testing. This setting is used throughout the article, unless otherwise noted. Full implementation details are given in the next section. We tested our methods on the well-known instances presented by Murray and Chu [24] for the FSTSP, which can be adapted to instances for the MFSTSP by increasing from one to $|D|$ the number of drones equipped on the truck. The 36 randomly generated instances have endurance $E = 20$ min or $E = 40$ min resulting in 72 instances, each with 10 customers located in an 8×8 area. In our experiments, the truck could carry from two to five drones. The depot is located in four possible positions: in the first position "a" the depot is almost in the barycenter of the customers; in "b", "c", and "d" it is close to the right border of the square, with a vertical position which is, respectively, in the barycenter, at the bottom border, and below the bottom border at a distance which is equal to the distance of the barycenter from this border. Moreover, the truck speed is 25 miles/h and based on Manhattan distances, the drones speed can be either 15, 25, 35 miles/h, and is based on Euclidean distances. We refer to Murray and Chu [24] for full details. We used the benchmark instances proposed by Murray and Chu to compare the proposed algorithms and select the best performing one. Afterward, we compared our best algorithm on a second set of instances proposed by Murray and Raj [25].

5.1 | Implementation

Formulations 4I-BC and 3I-BC required a branch-and-cut implementation, since they include TCS (31) and TCS2 (56), respectively, that are exponentially many. In Table 1, these constraints are summarized under the names TCSint and TCSfrac, when generated from integer and fractional solutions, respectively. To separate these constraints, we consider the residual graph $G' = (N, A')$ obtained from G by selecting the only arcs associated with a non zero variable (x, y, g) in the continuous relaxation of the model. We thus explore the graph starting from depot 0, until a truck path violating one of the constraints is identified, if any. During the same graph exploration, we also check if TBS (33) and TBS2 (57) are violated for fractional solutions (TBSfrac in Table 1) and, when a violation is found, we insert it (note that the elimination of backward sorties is always guaranteed by the time variables for integer solutions). In such a case, the overall cut separation procedure has a time complexity of $O(|A'|)$. We also include fractional *subtours elimination constraints* (SECfrac in Table 1) in all formulations. They are separated with the standard approach that requires to solve at most $O(n^2)$ max flow problems (see, e.g., [27]) on the residual graph. As branching strategy we used the CPLEX solver "strong branching," and an initial heuristic upper bound is provided to the solver.

In Table 1, one can see the number of cuts used in the branch-and-cut algorithms (up to optimality or when the 1 h time limit was reached) grouped with respect to different number of drones and type of formulation, averaged over all the 72 instances. We inserted one violated cut at a time in the order in which they are displayed in the table. We cut SECfrac first because the separating procedure is computationally less expensive than the procedure that identifies violated TCSfrac and TBSfrac cuts (which are detected in the same procedure). Preliminary tests guaranteed the profitability of the current implementation. In particular, for all formulations, the current setting improves the lower bound value at the root node by about 5%, on average,

TABLE 1 Average number of inserted cuts for type with respect to the number of drones and the used method on the 72 Murray and Chu instances

|D|	Form.	TCSint	SECfrac	TCSfrac	TBSfrac
2	4I-BC	55.86	6.78	9.26	21.08
	3I-BC	71.65	8.72	3.74	5.13
	4I-z	-	8.40	-	32.17
	3I-z	-	8.28	-	6.11
3	4I-BC	24.21	6.35	6.18	14.14
	3I-BC	41.88	7.17	3.39	3.44
	4I-z	-	7.13	-	16.19
	3I-z	-	7.01	-	3.19
4	4I-BC	10.78	5.90	5.65	11.92
	3I-BC	30.25	6.39	1.86	2.33
	3I-z	-	6.11	-	2.29
	4I-z	-	6.17	-	10.97
5	4I-BC	5.85	5.29	3.15	6.68
	3I-BC	12.56	5.10	0.97	1.07
	4I-z	-	5.35	-	5.01
	3I-z	-	5.21	-	1.17

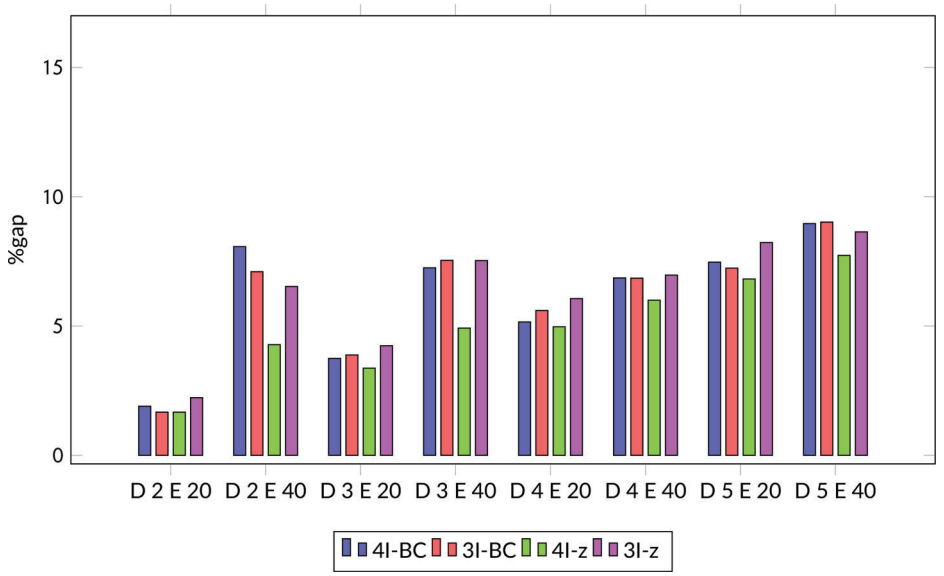


FIGURE 5 Percentage gap averaged over all instances computed for the four proposed algorithms. A comparison is shown with respect to the number of available drones D and the endurance E [Color figure can be viewed at wileyonlinelibrary.com]

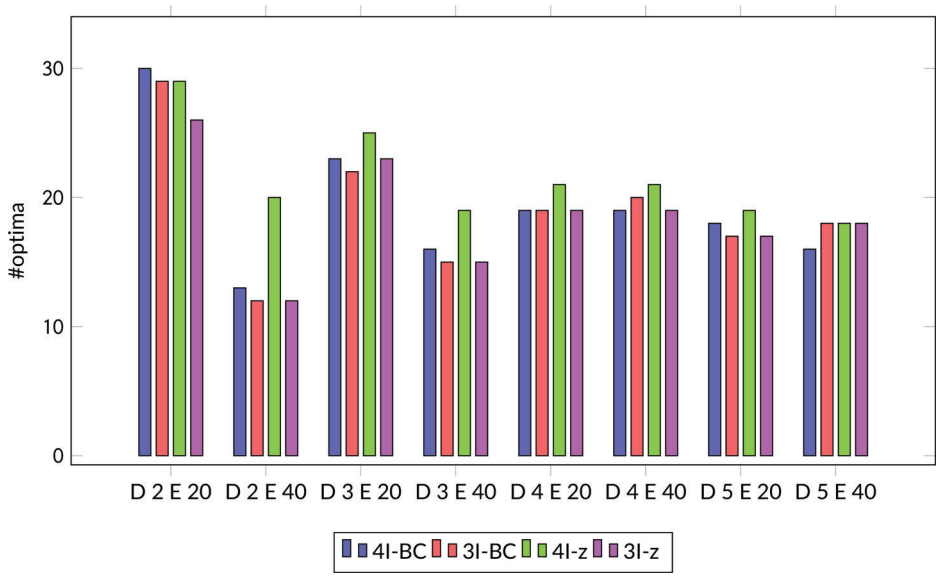


FIGURE 6 Number of optimal solutions obtained by the four proposed algorithms within the 1 h time limit. A comparison is shown with respect to the number of available drones D and the endurance E [Color figure can be viewed at wileyonlinelibrary.com]

with respect to the case with no cuts, being the SECfrac the cut providing the higher contribution among all. This justifies the choice of separating SECfrac first.

One can see that TCSint is the most used cut at the end of the iterations. However, the number of these cuts is relatively small, being that they are needed in formulations 4I-BC and 3I-BC to make the solution feasible. The number of needed TCSint is larger for the 3I-BC formulation than for the 4I-BC, while it is the opposite when it comes to the fractional TCSfrac. Also TBSfrac is shown to be relevant, being the second most important cut for 4I-BC and the first one for 4I-z and for 3I-z. A relevant number of SECfrac are inserted and its number is similar for all formulations. Moreover, one can see that the number of inserted cuts, especially the TCS ones, integer, and fractional, diminishes with the increase of the available drones, since infeasible crossing sorties are less likely to appear in a solution when a large number of drones is available.

5.2 | Numerical results for Murray and Chu benchmark

In this section, we present the results on the benchmark instances by Murray and Chu in order to decide which one of the proposed methods provides the best results. Note that the solutions for $|D|=1$ are treated in detail in other papers (see, e.g., Dell'Amico et al. [12]) and we do not report them in this work.

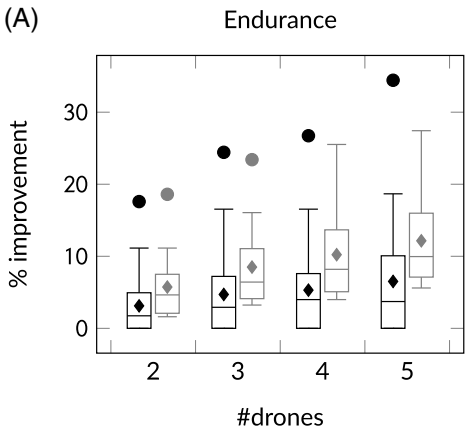
TABLE 2 Results for the 4I-z formulation per depot location

|D|	E	dep	%gap	time	opt	|D|	E	dep	%gap	time	opt
2	20	a	3.95	1376.21	6	4	20	a	4.47	2054.18	4
		b	0.73	668.60	8			b	4.39	1861.98	5
		c	1.13	736.17	8			c	6.37	1645.54	6
		d	0.85	985.21	7			d	4.66	1452.79	6
		avg/sum	1.67	941.55	29			avg/sum	4.97	1753.62	21
	40	a	5.27	2343.97	4		40	a	1.16	1290.87	7
		b	2.71	1559.32	6			b	4.21	1631.04	5
		c	3.80	1760.54	6			c	11.43	2018.69	4
		d	5.36	2017.14	4			d	7.20	2001.63	5
		avg/sum	4.28	1920.24	20			avg/sum	6.00	1735.56	21
	avg/sum		2.97	1430.89	49		avg/sum		5.49	1744.59	42
3	20	a	3.88	1668.48	6	5	20	a	5.39	1717.49	5
		b	2.99	1386.77	6			b	10.25	2058.28	4
		c	3.48	1115.18	7			c	7.13	1830.07	5
		d	3.11	1361.13	6			d	4.52	2077.75	5
		avg/sum	3.37	1382.89	25			avg/sum	6.82	1920.90	19
	40	a	2.60	2075.76	4		40	a	2.01	1466.69	6
		b	2.27	1511.27	6			b	9.60	2005.54	4
		c	7.54	1775.47	5			c	10.61	2006.31	4
		d	7.26	2007.41	4			d	8.69	2011.27	4
		avg/sum	4.92	1842.48	19			avg/sum	7.73	1872.45	18
	avg/sum		4.14	1612.68	44		avg/sum		7.28	1896.67	37

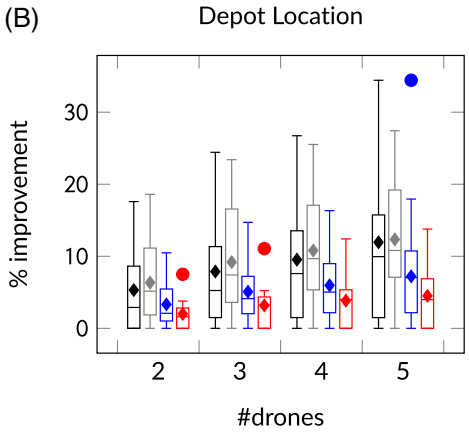
TABLE 3 Results for the 4I-z formulation per drone speed

|D|	E	speed	%gap	time	opt	|D|	E	speed	%gap	time	opt
2	20	15	0.00	107.32	12	4	20	15	1.10	1005.36	10
		25	4.29	2025.29	6			25	10.21	2445.39	4
		35	0.70	692.03	11			35	3.61	1810.12	7
		avg/sum	1.67	941.55	29			avg/sum	4.97	1753.62	21
	40	15	9.11	3083.75	3		40	15	10.64	2795.35	3
		25	2.00	1542.53	8			25	4.96	1486.33	8
		35	1.74	1134.45	9			35	2.39	924.99	10
		avg/sum	4.28	1920.24	20			avg/sum	6.00	1735.56	21
	avg/sum		2.97	1430.89	49		avg/sum		5.49	1744.59	42
3	20	15	0.57	551.73	11	5	20	15	2.68	1359.46	9
		25	7.50	2317.19	5			25	11.25	2512.42	4
		35	2.03	1279.75	9			35	6.54	1890.81	6
		avg/sum	3.37	1382.89	25			avg/sum	6.82	1920.90	19
	40	15	8.88	2757.01	4		40	15	12.30	3005.77	2
		25	3.13	1536.26	7			25	7.81	1403.94	8
		35	2.74	1234.15	8			35	3.07	1207.64	8
		avg/sum	4.92	1842.48	19			avg/sum	7.73	1872.45	18
	avg/sum		4.14	1612.68	44		avg/sum		7.28	1896.67	37

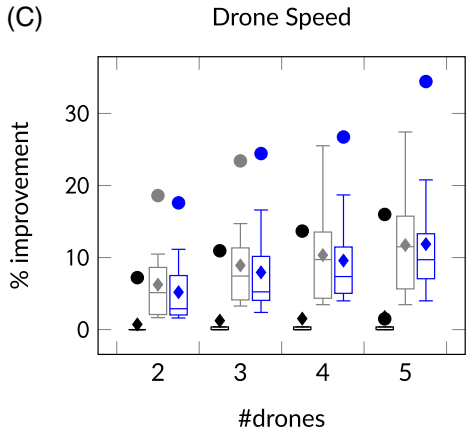
In Figures 5 and 6, we summarize, in two histograms, the results for the four proposed algorithms (4I-BC, 3I-BC, 4I-z, and 3I-z) by varying the number of available drones $|D|$ from two to five and the endurance E assuming the values of 20 and 40 min. In Figure 5, we show the average percentage gap computed as follows: $\%gap = 100 \cdot (UB - LB)/UB$ (where UB and LB are the upper and the lower bounds at the end of the iterations for each algorithm). In Figure 6, one can find the number of optimal solutions that each of the algorithms retrieved at the end of the iterations (either the optimal solution is found or the 1 h time limit is reached). One can notice that the average percentage gap increases with the number of available drones and by increasing the endurance. The first point is justified by the increase of the number of variables due to a larger number of available drones. The second one is due to the larger number of feasible sorties that are allowed by a larger endurance, which is linked to the trend of the number of optima found by the proposed algorithms, that follows the opposite direction. A clear conclusion that one can draw is that formulation 4I-z is the one providing the best results, and thus in the following we will consider only this one.



Percentage improvement with multiple drones and different endurance values: $E = 20$ in black, and $E = 40$ in gray



Percentage improvement with multiple drones and different depot locations: 'a' in black, 'b' in gray, 'c' in blue, and 'd' in red



Percentage improvement with multiple drones and different drone speeds: 15 mph in black, 25 mph in gray, and 35 mph in blue

FIGURE 7 Percentage improvement on the optimal solution values: the cases with multiple drones ($|D|$=2, … 5) with respect to the single drone case. Results grouped per endurance values in (A), per depot location in (B), and per drone speed in (C). All values are shown in a box-plot [Color figure can be viewed at wileyonlinelibrary.com]

Computational details are displayed in Tables 2 and 3. In Table 2, we show the results of formulation 4I-z for different number of available drones aggregated by endurance and depot location, while in Table 3, we show the same results aggregated by endurance and drone speed. In both tables we show the number of available drones $|D|$, the endurance E, the depot location *dep* or the drone speed *speed*, the averaged *%gap*, the computing time (averaged over all instances), having a 1 h time limit, and the number of optimal solutions obtained *opt*.

In Figure 7, we show the objective function percentage improvement (cost decrease) for the cases with two, three, four, and five available drones with respect to the single drone solution. These values are computed on the 35 instances that could be solved to optimality for every D. For each instance, we compute the gap as follows $100 \cdot (z_1 - z_{|D|})/z_1$, where z_1 is the objective function value of the single drone case and $z_{|D|}$ the case with $|D|$ available drones. We displayed the values in box-plots, in which the lower and the upper bounds are the first and the third quartile, the middle line shows the median, and the diamond shows the average. The points represent outliers, that are considered so, and thus depicted outside the whiskers, if they are above the the third quartile or below the first quartile by more than the interquartile range multiplied by 1.5.

The availability of multiple drones on the truck makes the solution cost decrease as the number of drones increases. Indeed, by increasing the number of drones, several sorties can be performed in parallel and this makes the objective function value decrease. Moreover, since the launches from the depot do not pay the launching time, the larger is the number of available drones the larger is the advantage of launching sorties from the depot. In Figure 7A, one can find the box-plot of the objective function percentage improvement when using multiple drones with difference values of endurance, $E = 20$ and $E = 40$. By increasing the number of available drones, the improvement becomes more relevant; however, it can be observed that a larger

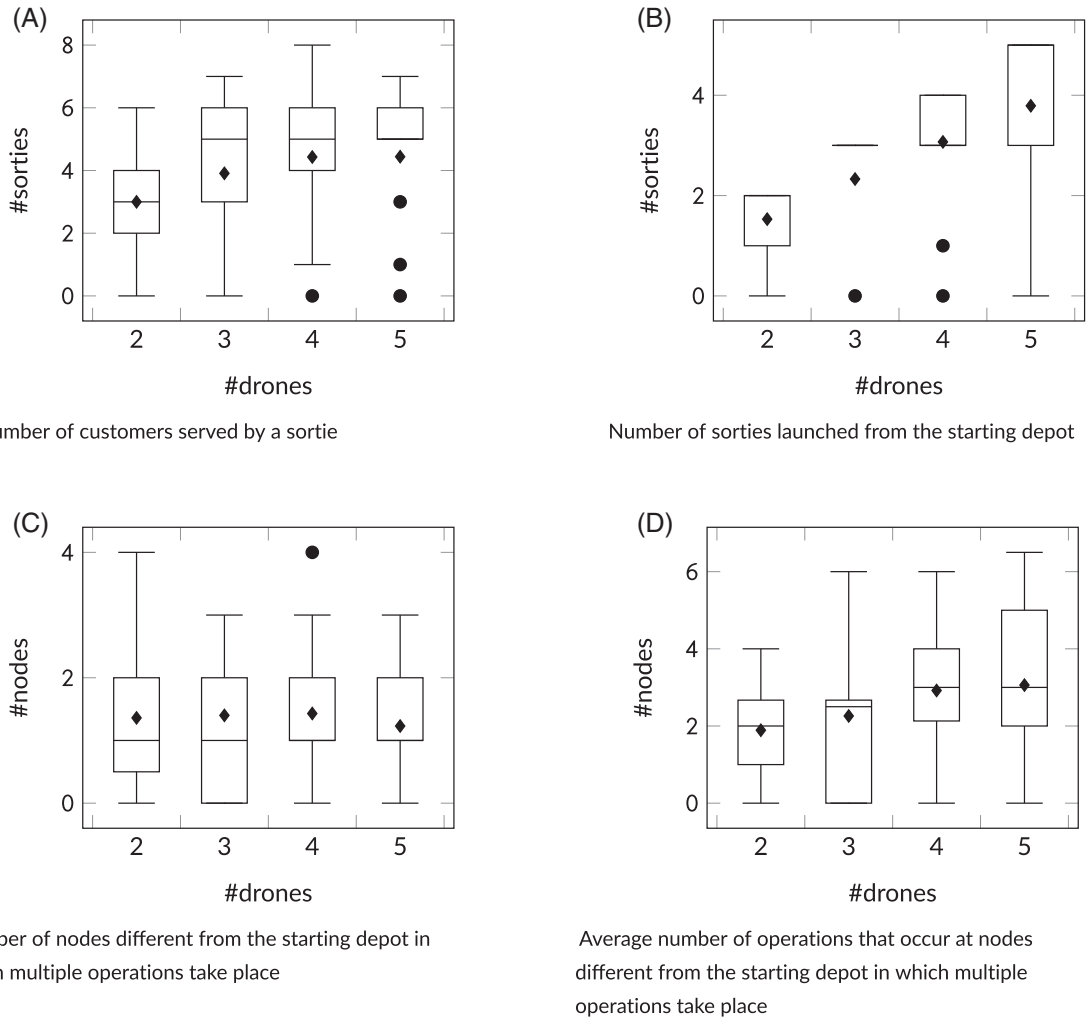


FIGURE 8 Study on the characteristics of the optimal solutions of the Murray and Chu instances, with 2 to 5 available drones

endurance such as $E = 40$ allows more sorties to become feasible, and the percentage improvement is significantly more relevant than with $E = 20$. Note that our results are similar to those obtained by Murray and Raj [25], that study instances with eight customers and with up to four available drones. In Figure 7B, we depict the percentage improvement of the objective function with respect to the depot location. When the depot is located in "a" and "b" (e.i., closer to the customers), by increasing the number of drones a larger number of sorties starting from and returning to the depot become feasible and the improvement on the objective function is more evident. On the other hand, when the depot is located slightly farther ("c" and even more "d") the improvement is more limited. In Figure 7C, the graph shows the behavior with respect to different drone speeds. The solution value almost does not improve when drones are slower, even if more drones are available: several sorties are infeasible because the drones are too slow to return to the truck within the battery endurance. The percentage improvement is much more consistent with the increase of the number of available drones when these are faster (25 and 35 mph speed).

In Figure 8, we show some insights on the optimal solution of the instances that could be solved to optimality. Note that the number of instances solved to optimality with different number of available drones can diverge. In Figure 8A, we show the number of customers served by a sortie. In Figure 8B, we show the number of sorties departing from the starting depot. In Figure 8C, we depict the number of nodes in which more than one operation happens excluding depot 0, for which no scheduling is needed. The average number of operations in those nodes is shown in Figure 8D. All values are displayed in a box-plot. One can see, as expected, that the number of customers served with a drone increases with the increase of the number of available drones. When $|D| = 5$, the number of sorties does not increase remarkably with respect to $|D| = 4$, since most of the nodes are already served by drones. The number of sorties launched from the starting depot increases with the number of available drones. This value is large because these sorties do not imply a launching cost and this justifies the study on the cases in which sorties from the depot imply a cost, as in Section 7. The number of nodes in which multiple operations happen, depot 0 excluded, increases when the number of available drones goes from two to four, but decreases with $|D| = 5$. However, this is justified by the fact that the average number of operations per node increases as shown in Figure 8D. The large number of nodes with multiple

TABLE 4 Computational results comparison between the standard setting and the improved one by running 4I-z on a selection of instances with $E = 20$, $|D| = 2, \ldots, 5$, and time limit of 1 h

	Standard		Improved	
|D|	time	%gap	time	%gap
2	941.55	1.67	148.32	0.00
3	1361.13	3.37	68.32	0.00
4	1753.62	4.97	320.54	0.00
5	1920.90	7.73	756.88	0.38

operations and the large number of operations that occur in those nodes show that the scheduling of the drone operations is relevant for the solution and justifies our study.

5.2.1 | Computational results using an improved setting

In order to better analyze the performances of our best method, namely, 4I-z, we performed additional tests on a randomly chosen subset of 12 instances of the Murray and Chu benchmark set, with $E = 20$ and $|D| = 2, \ldots, 5$ (including three instances for each depot location and four for each drone speed), by using a better performing machine with 3.6GHz i7-9700K CPU, CPLEX 12.10 as solver, and all the 8 threads. Table 4 compares the results obtained with our basic PC and the standard setting (single thread) to the results obtained with the fastest PC and the improved setting (eight threads). For both we show the average time in seconds and the average percentage gap. Note that all instances with $|D| = 2, 3, 4$ are solved to optimality and in short times, while the average gap of the instances with $|D| = 5$ within the time limit of 3600 s is only 0.38%. We can conclude that the MFSTPS is a challenging problem as other similar problems of the literature (see, e.g., [25] and [22]), but also that the proposed methods can provide good results with respect to them. In the remainder of this article, we continue to execute tests with our basic PC and the standard setting, to show data comparable with the previous section.

5.3 | Numerical results for Murray and Raj benchmark

We compared with the literature by testing the best performing method of ours, 4I-z, on the instances provided by Murray and Raj [25]. They use two set of instances, one with eight customers that they solve with an exact method and a second one with 10 customers that they solve with heuristic methods. Murray and Raj use different versions of endurance and different drone settings. We decided to use here the fixed endurance represented by a maximum amount of flight time of 700, 1400, 350, and 700 s, each linked to its corresponding drone setting, depending on the speed and range. This resulted in 80 instances for each number of available drones with eight customer, and 80 for each number of available drones with 10 customers. We also use

TABLE 5 Results on Murray and Raj instances with eight and 10 customers

c	|D|	%gap	%opt	time	MRtime
8	1	0.00	100	25.41	51.4
	2	0.00	100	227.30	1075.5
	3	2.44	85	829.19	2155.6
	4	4.25	65	1579.84	2430.5
	Buffalo	2.64	80	977.37	2349.1
	Seattle	0.71	95	353.50	507.4
	avg	1.67	88	665.44	1428.3
10	1	0.88	93	654.48	-
	2	2.95	78	1246.12	-
	3	8.07	53	2126.31	-
	4	7.87	46	2160.09	-
	Buffalo	3.45	69	1473.91	-
	Seattle	6.43	65	1619.59	-
	avg	4.94	67	1546.75	-

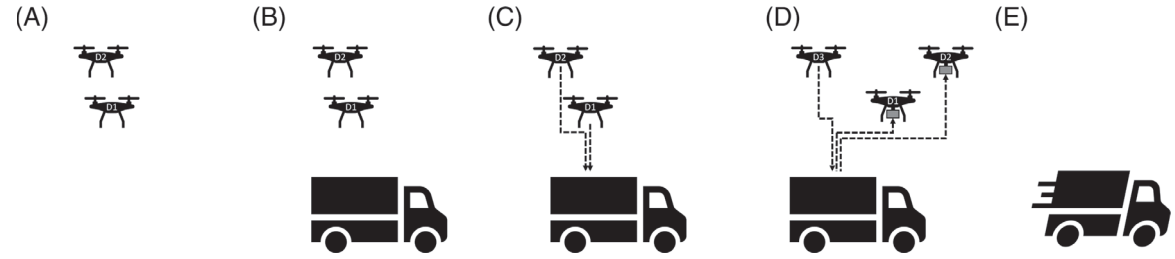


FIGURE 9 Sequence of operations of multiple drones and a truck that can occur at a certain node in the case in which no scheduling of operations is considered

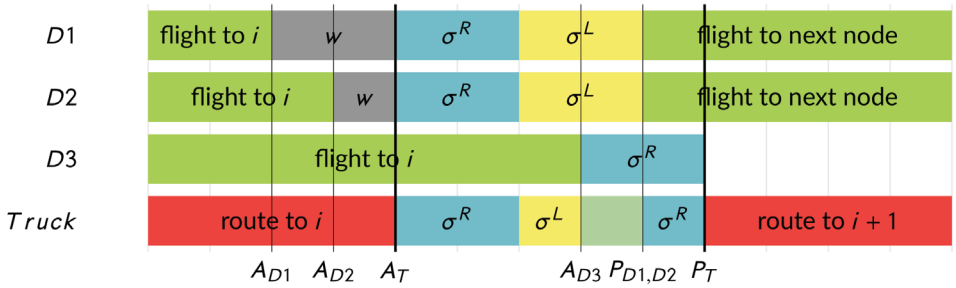


FIGURE 10 Example of multiple operations in a given node i for the variant with no scheduling of multiple operations is considered [Color figure can be viewed at wileyonlinelibrary.com]

a launching time of 60 s and a rendezvous time of 30 s. However, we solve a slightly different problem because we do not consider the service time when serving the customers, while Murray and Raj consider it in the scheduling phase. In Table 5, we show the average results over the 640 instances we run, reporting the number of customers, the number of available drones, the percentage gap, the solving time in seconds (having set a time limit of 1 h), and the percentage of optimal solutions obtained for each subset. We also show the time that Murray and Raj needed to solve the instances on a PC with an 8-core Intel i7-6700 processor and 16 GB RAM (*MRtime*). Note that their machine, compared to ours, is much faster according to [4]. The author declare that they could solve only the 66% of the eight customer instances, for which we could provide an optimal solution the 88% of the times. We underline once more that we solve a slightly different problem, but we believe that these results represent a meaningful comparison. One interesting fact is that the instances are divided into two sets called Buffalo and Seattle and Murray and Raj found easier to solve the Seattle instances, which we also do for the eight customer instances, while our method solves more easily the Buffalo instances with 10 customers.

6 | SCHEDULING VERSUS NO SCHEDULING

In this section, we compare the problem described in the previous part of this work, which we now refer to as *original model*, with the case in which no scheduling of drone operations is considered.

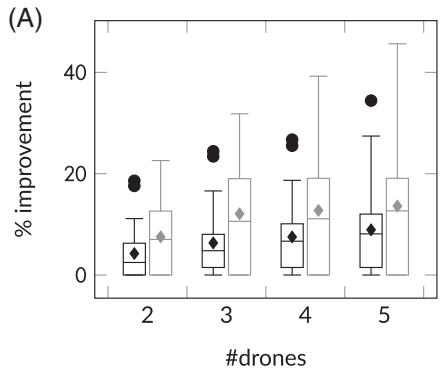
Recent works (see, e.g., Seifried [37]) consider the launching and rendezvous times included in the drone traveling times, and thus do not consider scheduling drone operations at the nodes in which multiple launches or rendezvous happen. This may lead to infeasible solutions in reality as shown in Section 3. However, we intend to investigate the variant in which parallel operations are allowed by technology. This can also be the case of non-aerial drones that can be sent from the truck and return to it with less restrictive limitations.

In Figure 9, one can see an example with multiple operations with no scheduling. In Figure 9A, drones $D1$ and $D2$ arrive before the truck and must wait until the truck arrives as in Figure 9B. At this point, the operations can start, but no scheduling is needed and both rendezvous can take place in parallel. The fact that the drone must return to the truck or be on the truck before it is launched is the only sequence constraint that must be guaranteed. Drones $D1$ and $D2$ can thus return to the truck and leave for a new sortie. The rendezvous of $D3$ can be done completely or partially in parallel with the launch of $D1$ and $D2$ depending on arrival time of drone $D3$. When the operations have been completed the truck can move to the next node on its route. The Gantt chart of this example can be found in Figure 10. The notation is the same used for Figure 3. Note that the launches of drones $D1$ and $D2$ partially overlap with the rendezvous of drone $D3$.

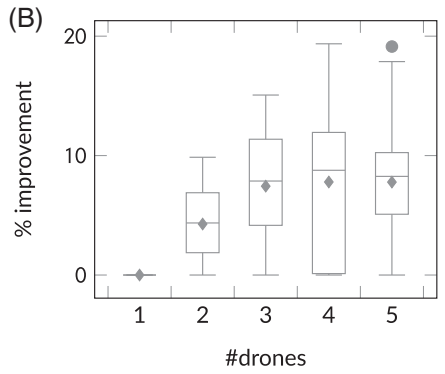
To account for this variant, we need to change the previously introduced formulations. Formulations 4I-BC and 4I-z can be rewritten without the following constraints: (19)–(29), and (40). The formulations 3I-BC and 3I-z can be rewritten without the following constraints: (19)–(22), (27), (40), and (50)–(55). Note that in case no scheduling is considered, there is no need to

TABLE 6 Results for the variant with no scheduling

|D|	E	%gap	time	opt
2	20	3.20	1553.38	26
	40	7.97	2591.42	17
	avg/sum	5.59	2072.40	43
3	20	5.91	1978.86	21
	40	9.63	2500.44	15
	avg/sum	7.77	2239.65	36
4	20	9.39	2711.42	15
	40	12.35	2932.91	11
	avg/sum	10.87	2822.16	26
5	20	10.85	2798.53	14
	40	14.80	2704.77	12
	avg/sum	12.82	2751.65	26



Percentage improvement on the optimal solution value with multiple drones ($|D| = 2, \ldots, 5$) with respect to the case with a single drone. We show the case with no scheduling (gray) and the model with scheduling (black) for a comparison



Percentage improvement on the optimal solution value of the case with no scheduling (gray) and a given number of available drones ($|D| = 1, \ldots, 5$) with respect to the model with scheduling

FIGURE 11 Percentage improvement of the no scheduling model with respect to the model including the scheduling of operations

include drone dependent variables in the formulations and the crossing sortie elimination constraints could also be simplified and provide improved computational results. However, that is not the scope of this publication. For this reason, the results of formulation 4I-z used to solve the model with no scheduling presents larger percentage gaps and less optimal solutions with respect to the scheduling case, as shown in Table 6.

In Figure 11A, for each instance, we compute the gap as follows $100 \cdot (z_1 - z_{|D|})/z_1$, where z_1 is the objective function value of the single drone case and $z_{|D|}$ is the multiple drone objective function value with $|D|$ drones. This is computed for all the instances for which the optimal solution value was at hand for all the number of available drones. In Figure 11A, one can note that the variant with no scheduling (gray) allows more drones to depart and serve customers and this makes the cost of the solution decrease by increasing the number of available drones. This variant shows large improvements when increasing the number of drones, reaching up to more than 13% improvement on average in the case with $|D| = 5$, with respect to the single drone case. This is justified by the fact that many drones can be launched and collected in parallel.

In Figure 11B, we show the percentage improvement between the solution value of the original model (z_{sched}) and the variant with no scheduling ($z_{no-sched}$) averaged over all the instances for which we obtained the optimal solution. For each drone number and instance, the improvement is computed as $100 \cdot (z_{sched} - z_{no-sched})/z_{sched}$. In Figure 11B, one can see that the variant with no scheduling provides solution values that largely improve the original model, since multiple operations can be done in parallel and this produces less costly solutions. The increase of the solution value decelerates when increasing the number of available drones, since the maximum number of customers effectively serviced with drones has already been reached for many instances. The values for $|D| = 5$ appear lower than those of $|D| = 4$ because the baseline is reduced, given that the number of instances for which the optimal solution was available (over which we computed the gap) is lower. However, the lower bound of the box-plot for $|D| = 5$ is largely higher than that of $|D| = 4$. This significant difference between the values in the optimal solutions of the original model and those of the case with no scheduling strongly justifies our interest in the scheduling component.

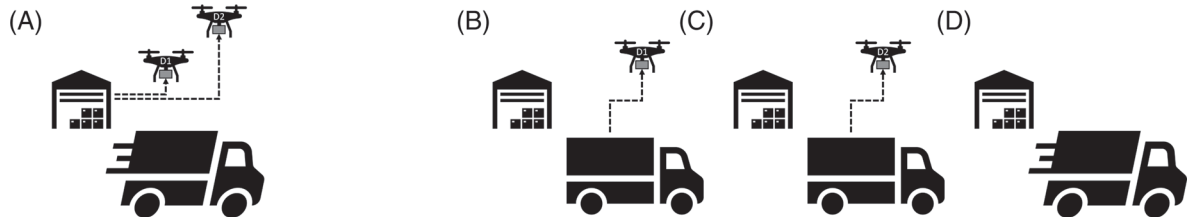


FIGURE 12 Example of two drones starting their sortie at the depot: in the original model (A) no launching time is considered at the depot because the drones and the truck can leave uncoupled and in parallel; in the variant in which the launching time from the depot is considered, the truck is stationary at the depot while the two drones are launched in sequence as in (B) and (C), and can leave only in (D)

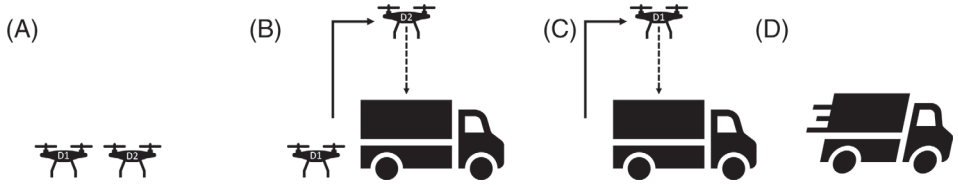


FIGURE 13 Sequence of operations of two drones and the truck that can occur at a certain node in the case in which drones are allowed to wait on the ground

7 | OTHER VARIANTS OF THE ORIGINAL MODEL

In this section, we introduce other two variants of the original model. The first variant considers the launching time also when the drones are launched from the depot. One should note that we decided to follow the literature and consider no launching time at the depot, since the truck and the drone can leave the depot uncoupled. On the other hand, when increasing the number of drones, many drones are launched from the depot in the optimal solution, and this implies an advantage in terms of total time and thus in the objective function value. It is interesting to evaluate the influence of this component on the solution. In Figure 12 we show an example of two drones ($D1$ and $D2$) that are launched from the depot. Figure 12A shows the case of the original model, in which the drones and the truck are uncoupled at the starting depot and thus the drones leave in parallel among them and the truck, but never before the truck. In Figures 12B, and 12C, one can see that the drones leave from the truck also at the depot and that the truck must wait for the launching times of both $D1$ and $D2$ to leave as in Figure 12D. Moreover, the launching sequence (the only scheduling that can occur in the starting depot) is taken into account.

It is straightforward to account for this variant in the proposed formulations. For formulations 4I-BC and 4I-z, we need to remove constraints (16) and (34) and change the following constraints, rewriting them for all $i \in N$ instead of N_+: (17), (19)–(22), (35). For 3I-BC and 3I-z, we need to remove constraints (16) and (34) and change the following constraints (19)–(22), (35), (48)–(49), rewriting them for all $i \in N$ instead of N_+.

The second variant that we investigate allows the drones to wait on the ground to save energy. This variant has already been considered in the literature (see, e.g., Phan et al. [40]) and also for the single drone case (see, e.g., Ponza [32] and Dell'Amico et al. [12]). In Figure 13, we provide an example. In Figure 13A two drones, $D1$ and $D2$, arrive at the rendezvous node before the truck does and land on the ground to save battery. In Figure 13B, the truck arrives and one of the two drones ($D2$) starts its rendezvous operation, while $D1$ keeps on waiting on the ground. In Figure 13C, $D1$ can start its rendezvous with the truck and in Figure 13D, the truck can leave for the subsequent node on its route.

To include this variant in the formulations, we only need to modify constraints (13) and (14) in 4I-BC and 4I-z and (46) and (47) in 3I-BC and 3I-z.

A comparison among the different variants proposed is given in Figure 14A, where we display the percentage difference in the objective function for the multiple drones case with respect to the single drone one, for all the considered variants. Similar to what was done in Section 4, for each variant and instance we compute the gap as follows $100 \cdot (z_1 - z_{|D|})/z_1$, where z_1 is the objective function value of the single drone case and $z_{|D|}$ is the multiple drone case with $|D|$ available drones. This is computed for all the instances for which the optimal solution value was at hand for all the number of available drones. These gaps are displayed in box-plots. In black, one can find the original model, in blue the case with launching time from the depot, and in red the wait case. One can see that most of the times the increment of the number of available drones leads to an improvement in the objective function value. Indeed, the larger is the number of the available drones, the more the possibilities to serve customers in parallel are and the less the total service time is. The original model (black in the figure) follows this pattern. The variant in which the launches from the depot must pay the launching time (blue in the figure) shows that this component is very important, since by increasing the number of available drones, the solution value does not improve remarkably as for the other variants, that we recall take advantage of multiple drones departing from the depot without paying the launching time. One can see that

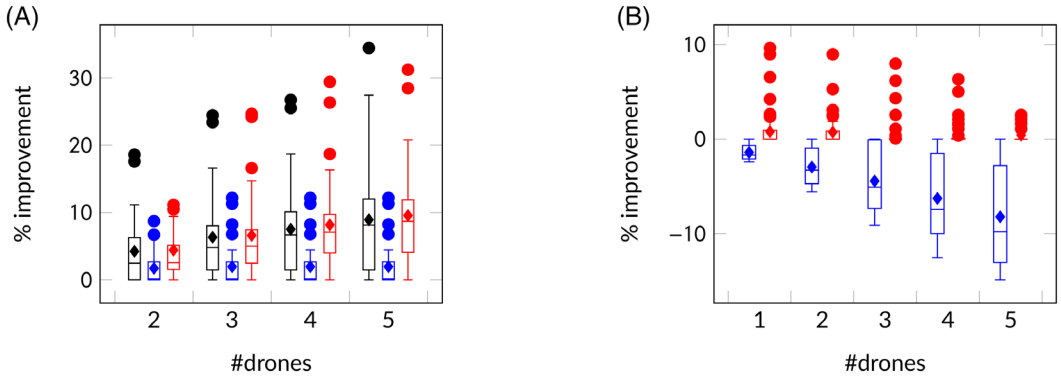


FIGURE 14 Comparison among the variants of the model: the original model (black), no launching time from the depot (in blue), waiting on the ground allowed (in red). (A) Percentage improvement on the optimal solution value for each of the considered variants with respect to the corresponding case with a single drone. (B) Percentage improvement on the optimal solution value: the solution of each variant with respect to the original model with the same number of drones [Color figure can be viewed at wileyonlinelibrary.com]

having more than three drones available usually leads to no improvement for the instances we solved to optimality, but this is not necessarily true for every instance. The wait case (in red in the figure) has a behavior similar to the original model, but the improvement in the objective function is more evident, especially in the lower bound of the box-plot. This can be justified by the fact that more sorties become feasible when the drone can wait on the ground to save battery and thus less costly solutions become feasible.

In Figure 11, we show the percentage improvement between the solution value of the original model ($z_{original}$) and each one of the other analyzed variants ($z_{variant}$). For each drone number and instance, the improvement is computed as $100 \cdot (z_{original} - z_{variant})/z_{original}$.

One can observe that the percentage improvement of the version with launching time at the depot decreases strongly with the increase of the number of available drones. This is due to the fact that the original model takes advantage of multiple depot departures, while launching multiple drones from the depot is less favorable for this variant. This difference between the cost of the two versions is more and more visible with the increase of the number of available drones. The solutions of the wait variant have a lower cost than those of the original model. However, the improvement is small and slowly decreases with the increase of the number of available drones. In particular, when adding more available drones, the gap between the solution values of the two variants is zero most of the times. This means that increasing the number of available drones in the original model can compensate the lack of possibility of waiting, and this produces solutions similar to those of the wait variant. Note also that the outlier values decrease as well with the increase of the number of available drones.

8 | CONCLUSIONS

In this article, we considered the version of the Flying Sidekick Traveling Salesman Problem with multiple drones, where a truck is equipped with a set of identical drones and all can work together to serve a set of customers. In the version studied, the drones need time to be launched and collected when the truck is stationary at some node of the considered network and the scheduling of launching and rendezvous operations of multiple drones is an important component of the problem. To solve the problem, we proposed four novel formulations implemented as branch-and-cut algorithms that improved upon the related literature. We studied the effect of having up to five available drones on the truck and with up to 10 customers to serve. We also considered three variants of the problem and evaluated the influence of each of these on the solution. This kind of problems is very challenging even when it comes to solve small size instances, and we believe there is space for new improvements; in particular, we think that future works should focus on the scheduling component of this problem.

ORCID

Stefano Novellani https://orcid.org/0000-0001-8600-8000

REFERENCES

[1] Coronavirus: Grocer uses robots to delivery grocery orders amid COVID-19 pandemic, 2020, May 29, 2020, available at https://6abc.com/shopping/grocer-using-delivery-robots-during-coronavirus-pandemic/6060134/.

[2] Drone delivery Canada asks for COVID-19 use cases, 2020, May 29, 2020, available at https://www.gpsworld.com/drone-delivery-canada-asks-for-covid-19-use-cases/.

[3] Here's what the uber eats delivery drone looks like, 2020, May 29, 2020, available at https://techcrunch.com/2019/10/28/heres-what-the-uber-eats-delivery-drone-looks-like/.

[4] Userbenchmark, 2020, May 29, 2020, available at https://cpu.userbenchmark.com/.

[5] Zipline will bring its medical delivery drones to the u.s. to help fight the coronavirus, 2020, May 29, 2020, available at https://www.fastcompany.com/90483592/zipline-will-use-its-medical-delivery-drones-to-the-u-s-to-help-fight-the-coronavirus.

[6] N. Agatz, P. Bouman, and M. Schmidt, *Optimization approaches for the traveling salesman problem with drone*, Transp. Sci. **52** (2018), 965–981.

[7] P. Bouman, N. Agatz, and M. Schmidt, *Dynamic programming approaches for the traveling salesman problem with drone*, Networks **72** (2018), 528–542.

[8] Y. S. Chang and H. J. Lee, *Optimal delivery routing with wider drone-delivery areas along a shorter truck-route*, Expert Syst. Appl. **104** (2018), 307–317.

[9] R. Daknama and E. Kraus, Vehicle routing with drones, 2017, arXiv preprint arXiv:1705.06431.

[10] J. C. de Freitas and P. H. V. Penna, *A randomized variable neighborhood descent heuristic to solve the flying sidekick traveling salesman problem*, Electron Notes Discrete Math. **66** (2018), 95–102.

[11] J. C. de Freitas and P. H. V. Penna, *A variable neighborhood search for flying sidekick traveling salesman problem*, Int. Trans. Oper. Res. **27** (2020), 267–290.

[12] M. Dell'Amico, R. Montemanni, and S. Novellani, *Drone-assisted deliveries: New formulations for the flying sidekick traveling salesman problem*, Optim. Lett. (2019), 1–32. https://doi.org/10.1007/s11590-019-01492-z.

[13] M. Dell'Amico, R. Montemanni, and S. Novellani, Models and algorithms for the flying sidekick traveling salesman problem, 2019, arXiv preprint arXiv:1910.02559.

[14] M. Dell'Amico, R. Montemanni, and S. Novellani, *A random restart local search matheuristic for the flying sidekick traveling salesman problem*, Proceedings of the 2021 8th International Conference on Industrial Engineering and Applications, Europe, 2021.

[15] Q. M. Ha, Y. Deville, and Q. D. Pham, M. H. Hà, *On the min-cost traveling salesman problem with drone*, Transp. Res. Part C Emerg. Technol. **86** (2018), 597–621.

[16] Q. M. Ha, Y. Deville, Q. D. Pham, and M. H. Hà, *Heuristic methods for the traveling salesman problem with drone*, Comput. Sci. **arXiv** (2015), 1509.08764v1.

[17] M. Hu, W. Liu, J. Lu, R. Fu, K. Peng, X. Ma, and J. Liu, *On the joint design of routing and scheduling for vehicle-assisted multi-uav inspection*, Futur. Gener. Comput. Syst. **94** (2019), 214–223.

[18] M. Hu, W. Liu, K. Peng, X. Ma, W. Cheng, J. Liu, and B. Li, *Joint routing and scheduling for vehicle-assisted multi-drone surveillance*, IEEE Internet Things J. **6** (2018), 1781–1790.

[19] M. Joerss, J. Schröder, F. Neuhaus, C. Klink, and F. Mann, *Parcel delivery - the future of last mile*, Travel. Transp. Logist. (2016). https://www.mckinsey.com/~/media/mckinsey/industries/travel%20transport%20and%20logistics/our%20insights/how%20customer%20demands%20are%20reshaping%20last%20mile%20delivery/parcel_delivery_the_future_of_last_mile.ashx.

[20] A. Karak and K. Abdelghany, *The hybrid vehicle-drone routing problem for pick-up and delivery services*, Transp. Res. Part C Emerg. Technol. **102** (2019), 427–449.

[21] P. Kitjacharoenchai, M. Ventresca, M. Moshref-Javadi, S. Lee, J. M. Tanchoco, and P. A. Brunese, *Multiple traveling salesman problem with drones: Mathematical model and heuristic approach*, Comput. Ind. Eng. **129** (2019), 14–30.

[22] M. Moshref-Javadi, A. Hemmati, and M. Winkenbach, *A truck and drones model for last-mile delivery: A mathematical model and heuristic approach*, Appl. Math. Model. **80** (2020), 290–318.

[23] M. Moshref-Javadi, S. Lee, and M. Winkenbach, *Design and evaluation of a multi-trip delivery model with truck and drones*, Transp. Res. Part E Logist. Transp. Rev. **136** (2020), 101887.

[24] C. C. Murray and A. G. Chu, *The flying sidekick traveling salesman problem: Optimization of drone-assisted parcel delivery*, Transp. Res. Part C Emerg. Technol. **54** (2015), 86–109.

[25] C. C. Murray and R. Raj, *The multiple flying sidekicks traveling salesman problem: Parcel delivery with multiple drones*, Transp. Res. Part C Emerg. Technol. **110** (2020), 368–398.

[26] A. Otto, N. Agatz, J. Campbell, B. Golden, and E. Pesch, *Optimization approaches for civil applications of unmanned aerial vehicles (uavs) or aerial drones: A survey*, Networks **72** (2018), 411–458.

[27] M. Padberg and G. Rinaldi, *Facet identification for the symmetric traveling salesman polytope*, Math. Program. **47** (1990), 219–257.

[28] K. Peng, J. Du, F. Lu, Q. Sun, Y. Dong, P. Zhou, and M. Hu, *A hybrid genetic algorithm on routing and scheduling for vehicle-assisted multi-drone parcel delivery*, IEEE Access **7** (2019), 49191–49200.

[29] S. Poikonen and B. Golden, *Multi-visit drone routing problem*, Comput. Oper. Res. **113** (2020), 104802.

[30] S. Poikonen, B. Golden, and E. A. Wasil, *A branch-and-bound approach to the traveling salesman problem with a drone*, INFORMS J. Comput. **31** (2019), 335–346.

[31] S. Poikonen, X. Wang, and B. Golden, *The vehicle routing problem with drones: Extended models and connections*, Networks **70** (2017), 34–43.

[32] A. Ponza, Optimization of drone-assisted parcel delivery, Master's thesis, Univ Degli Studi Di Padova, 2016.

[33] L. D. P. Pugliese, F. Guerriero, D. Zorbas, and T. Razafindralambo, *Modelling the mobile target covering problem using flying drones*, Optim. Lett. **10** (2016), 1021–1052.

[34] D. Sacramento, D. Pisinger, and S. Ropke, *An adaptive large neighborhood search metaheuristic for the vehicle routing problem with drones*, Transp. Res. Part C Emerg. Technol. **102** (2019), 289–315.

[35] M. Salama and S. Srinivas, *Joint optimization of customer location clustering and drone-based routing for last-mile deliveries*, Transp. Res. Part C Emerg. Technol. **114** (2020), 620–642.

[36] D. Schermer, M. Moeini, and O. Wendt, *A matheuristic for the vehicle routing problem with drones and its variants*, Transp. Res. Part C Emerg. Technol. **106** (2019), 166–204.

[37] K. Seifried, The traveling salesman problem with one truck and multiple drones, 2019.

[38] B. Skorup and C. Haaland, How drones can help fight the coronavirus, 2020, May 29, 2020, available at https://www.mercatus.org/publications/covid-19-policy-brief-series/how-drones-can-help-fight-coronavirus.

[39] Statista Retail e-commerce sales worldwide from 2014 to 2021 (in billion u.s. dollars), 2020, May 29, 2020, available at https://www.statista.com/statistics/379046/worldwide-retail-e-commerce-sales/.

[40] P. A. Tu, N. T. Dat, and P. Q. Dung, *Traveling salesman problem with multiple drones*. Proceedings of the 9th International Symposium on Information and Communication Technology; Danang City, Vietnam; 2018. pp. 46–53.

[41] X. Wang, S. Poikonen, and B. Golden, *The vehicle routing problem with drones: Several worst-case results*, Optim. Lett. **11** (2017), 679–697.
[42] Z. Wang and J. B. Sheu, *Vehicle routing problem with drones*, Transp. Res. Part B Methodol. **122** (2019), 350–364.
[43] R. Wolleswinkel, V. Lukic, W. Jap, R. Chan, J. Govers, and S. Banerjee, An onslaught of new rivals in parcel and express, BCG Website, 2018.
[44] E. E. Yurek and C. H. Ozmutlu, *A decomposition-based iterative optimization algorithm for traveling salesman problem with drone*, Transp. Res. Part C Emerg. Technol. **91** (2018), 249–262.

How to cite this article: Dell'Amico M, Montemanni R, Novellani S. Modeling the flying sidekick traveling salesman problem with multiple drones. *Networks.* 2021;1–25. https://doi.org/10.1002/net.22022